

Trabajo de Fin de Máster

APRENDIZAJE MULTITAREA PARA DETECCIÓN DE CÁNCER EN MAMOGRAFÍAS

MULTI-TASK LEARNING FOR BREAST CANCER DETECTION IN MAMMOGRAPHIES

Autor: **Fernández Barrill, Íñigo**

Tutores: García Domínguez, Manuel

Inés Armas, Adrián

MÁSTER

**MÁSTER EN CIENCIA DE DATOS Y APRENDIZAJE
AUTOMÁTICO**

ESCUELA DE MÁSTER Y DOCTORADO



**UNIVERSIDAD
DE LA RIOJA**

AÑO ACADÉMICO: 2022/2023

Resumen

El cáncer de mama es una gran preocupación a nivel global, y la detección temprana juega un papel crucial en el resultado final. La herramienta más utilizada para detectarlo es la mamografía, que ofrece información precisamente en las etapas iniciales. En este trabajo, se propone utilizar una arquitectura de Aprendizaje Multi Tarea para mejorar los resultados en la detección de cáncer de mama.

Esta arquitectura aprovecha el uso de varias tareas relacionadas entre sí para mejorar el rendimiento frente a hacer las mismas tareas de manera individual. El problema principal es la detección de cáncer, y para ello se puede o bien realizar tareas secundarias que consistan en calcular la edad y la densidad de la mama del paciente, o bien extraer las características de las mamografías y junto a otros datos sobre ellas, predecir el diagnóstico del cáncer de mama.

Este trabajo aplica las técnicas del Aprendizaje Multi Tarea al problema concreto de la detección del cáncer de mama en mamografías, para comprobar si se obtiene un beneficio respecto al uso de técnicas individuales.

Abstract

Breast cancer is a significant global concern, and early detection plays a crucial role in the final result. Mammography is the most used tool for its detection, it offers information precisely at its early stages. In this study, it's proposed a Multi-Task Learning architecture for enhancing the results of breast cancer detection.

This architecture takes advantage of the use of multiple related tasks to improve the performance of those same tasks individually. The main problem is cancer detection, and it can be done either secondary tasks as figure out the age and breast's density, or to extract the mammographie's features and along with other related data, predict the breast cancer's diagnosis.

This study applies Multi-Task Learning techniques to the particular problem of breast cancer detection in mammographies, to check if there's a benefit in comparison to the use of individual tasks.

Índice

1. Introducción	4
1.1. Contexto	4
1.2. Motivación	5
1.3. Objetivos	6
1.4. Librerías y herramientas	6
2. Marco teórico	8
2.1. Tarea de clasificación	8
2.2. Inteligencia Artificial, Aprendizaje Automático y Aprendizaje Profundo . . .	9
2.3. Modelos y arquitecturas	10
2.4. Métodos de evaluación	13
3. Análisis de los datos	16
3.1. Conjunto de datos	16
3.2. Reducción de volumen	17
3.3. Limpieza del conjunto de datos	18
3.4. División del conjunto de datos	19
4. Evaluación inicial del problema	21
4.1. Modelos para imagen. Parte 1	21
4.2. Balanceo de datos	23
4.2.1. Balanceo por pacientes	24
4.2.2. Balanceo por imágenes	24
4.3. Modelos para imagen. Parte 2	24
4.4. Modelos para datos. Parte 1	26

4.5. Selección de características	28
4.6. Modelos para datos. Parte 2	30
5. Aproximaciones Multi Tarea	32
5.1. Primera aproximación	32
5.2. Segunda aproximación	38
5.2.1. Descripción de la arquitectura Multi Tarea de la segunda aproximación	39
5.2.2. Resultados de la arquitectura Multi Tarea de la segunda aproximación	43
6. Conclusiones	45
7. Bibliografía y referencias	46
A. Descripción del conjunto de datos	48
A.1. Descripción del conjunto completo	48
A.2. Descripción del conjunto tras la limpieza	50
A.3. Descripción del conjunto balanceado	53
B. Resultados sobre el balanceo por pacientes	55
C. Ajuste de parámetros para los modelos de datos	56
D. Modelo Multi Tarea, Aproximación 2 con función de pérdida única	58

1. Introducción

1.1. Contexto

El cáncer es una de las principales causas de muerte a nivel mundial, en 2020 se diagnosticaron aproximadamente 18,1 millones de casos, y las previsiones estiman un aumento en las próximas dos décadas hasta los 27 millones de casos. Del mismo modo, la mortalidad de esta afección en 2020 fue de casi 10 millones de personas y se estima que aumente hasta los 16,3 millones para el año 2040. En cuanto al porcentaje de pacientes que logran superar la enfermedad este es de un 55,3 % en hombres y un 61,7 % en mujeres [10].

Se trata de una enfermedad en la que se desarrollan células con anomalías que se multiplican y destruyen el tejido corporal. Hay sobre 100 tipos distintos de cáncer, con distintos síntomas y gravedades. Las principales causas de desarrollo de cáncer son el consumo de tabaco y los hábitos poco saludables. Pese a que hay enfermedades más mortales, a nivel social el cáncer tiene una fama más temible, pues su tratamiento es difícil y puede dejar secuelas graves. De hecho, es de las pocas afecciones que tienen un día mundial dedicado a él (4 de Febrero) y a su tratamiento se le llama comúnmente "Lucha contra el cáncer". También es la enfermedad a nivel mundial con más dinero destinado a combatirla.

De todos los tipos de cáncer, el más común es el de mama, que se desarrolla en el revestimiento de los órganos mamarios, principalmente en los conductos que transportan la leche desde la mama hasta el pezón y de los órganos que se encargan de producir la leche, que se llaman lobulillos. Este tipo de cáncer afecta principalmente a mujeres (más del 99 % de los casos se producen en mujeres [1]). El cáncer de mama, pese a ser muy común, tiene una tasa de mortalidad baja, sin embargo, sus consecuencias son notables. Además, a pesar de tener una tasa de mortalidad baja, en los países occidentales es la primera causa de muerte en mujeres, la quinta a nivel global [1] [21].

En este trabajo se ha decidido centrarse en el cáncer de mama, pues su prevención y detección temprana reduce significativamente el riesgo que presenta esta enfermedad. Si se detecta en las etapas iniciales el tratamiento es más sencillo y se puede llegar a mitigar la enfermedad antes de que ésta presente síntomas.

En medicina las técnicas más extendidas para la detección del cáncer de mama son las mamografías, el uso de ecografías y la resonancia magnética, siendo la más común la primera de ellas. Una mamografía es una imagen de la mama realizada mediante rayos X que ayuda a identificar posibles signos de cáncer de mama en sus etapas iniciales. También puede usarse para estudiar mejor el caso concreto una vez diagnosticado el cáncer. Dentro de

una mamografía los principales signos que se buscan son zonas que presenten una densidad mayor a la vista en el resto del tejido. Esto en la propia imagen se aprecia en zonas más claras dentro de las mamas. Si bien todas estas técnicas aportan información visual, este trabajo busca complementar esa información con datos sobre el paciente y dar una respuesta con más contexto. También, el proceso de análisis de estas imágenes se hace de forma manual y requiere que haya un médico disponible. Es por eso por lo que puede acarrear retrasos, y un modelo basado en Inteligencia Artificial puede ayudar a solventar estos problemas y a acelerar el proceso.

1.2. Motivación

En los últimos años, con el avance de la Inteligencia Artificial, se han desarrollado muchas soluciones que permiten realizar procesos costosos y repetitivos de forma más sencilla. Algunos ejemplos de estos trabajos pueden ser el análisis de documentos mediante Reconocimiento Óptico de Caracteres (como los modelos que extraen texto de imágenes, normalmente basados en *Tesseract* [14]), el uso de ChatBots para la atención al cliente (como los asistentes de algunas redes sociales [23]) o la clasificación de textos mediante bolsas de palabras (como los detectores de spam en correos electrónicos [3]). El campo que más se está beneficiando actualmente de este desarrollo es el industrial, donde estas soluciones ayudan a optimizar la cadena de procesos.

Concretamente, en el ámbito de la medicina, el Aprendizaje Automático también está cada vez más extendido y ha abierto nuevas vías en la investigación de patógenos. Técnicas basadas en este campo se utilizan sobre todo en el diagnóstico (esto engloba la detección y la predicción), ya que un modelo tiene una mayor capacidad para manejar datos y es más eficiente, rápido y preciso a la hora de detectar patologías que un humano. De todas formas, esta tecnología precisa de supervisión humana, y más que un reemplazo, es siempre una herramienta que complementa y mejora el trabajo de las personas.

Es por eso, por lo que una herramienta de este tipo puede resultar muy útil en el diagnóstico de una enfermedad como puede ser el cáncer, cuyo factor más determinante para bajar la mortalidad es una detección temprana. Actualmente las aproximaciones tecnológicas para este problema suelen estar basadas en el estudio y análisis de imágenes de mamografía usando redes convolucionales. Sin embargo, no se tienen en cuenta el resto de información biológica del paciente.

En el campo del Aprendizaje Automático cada vez se utilizan más las soluciones basadas en modelos Multi Tarea. Esto consiste en entrenar un modelo para que realice varias tareas relacionadas entre ellas de manera simultánea. Al enfocar el aprendizaje de esta manera, el

modelo aprende las características comunes a todas las tareas, lo que resulta en un mejor rendimiento en comparación a la realización de estas tareas por separado. También, este procedimiento ayuda a la generalización del modelo y evita el sobreaprendizaje. En el caso concreto de este trabajo, como hasta ahora el cáncer de mama se diagnostica únicamente con la información que dan las imágenes (en este caso, las mamografías), al poder usar también la información asociada a los datos biológicos (como, por ejemplo, la edad) usando un modelo Multi Tarea, que permite utilizar toda esa información, se puede mejorar el diagnóstico y acelerarlo.

1.3. Objetivos

Ya que el cáncer de mama es un gran problema, es muy importante que se estudien y se mejoren los procedimientos para su prevención, detección y tratamiento. También, teniendo en cuenta la situación actual en los hospitales, es necesario que se ayude al personal sanitario a agilizar la atención y mejorar su calidad.

Hay que tener en cuenta que debe haber un equilibrio a la hora de realizar el diagnóstico, pues si un caso positivo no se diagnostica, las consecuencias son graves, desde la muerte de la persona hasta un tratamiento más complicado al haber detectado la enfermedad en un estado más avanzado. En caso contrario, si se emite un falso positivo, también puede ser muy perjudicial, pues los tratamientos pueden ser caros e invasivos, además de las posibles secuelas que estos tratamientos puedan dejar en la persona.

Por ello, el objetivo de este trabajo es estudiar las técnicas de Aprendizaje Multi Tarea para aplicarlas a la detección de cáncer en mamografías. Dicho de otra manera, el objetivo es realizar una clasificación para distinguir aquellas instancias que presentan cáncer de mama de las que no. Para ello, se van a estudiar diferentes modelos de Aprendizaje Profundo y comparar los resultados de esos modelos con sus respectivas implementaciones en una aproximación Multi Tarea, para ver si se pueden obtener beneficios al usar este tipo de técnicas.

1.4. Librerías y herramientas

Este trabajo se ha realizado usando el lenguaje de programación *Python*, que es el más extendido para tareas relacionadas con el Aprendizaje Automático y cuenta con un amplio abanico de librerías especializadas en este campo. Concretamente, este trabajo se va a apoyar en las siguientes librerías:

- *FastAI* [2]: Orientada a Aprendizaje Automático de alto nivel basada en *PyTorch* [12]. Es muy utilizada debido a su sencillez y potencia y a que permite realizar todas las tareas necesarias para crear y entrenar los modelos.
- *Timm* [15]: Proporciona una colección de distintas arquitecturas y modelos preentrenados en *PyTorch* [12], para tareas relacionadas principalmente con la visión por computador y el Aprendizaje Automático.
- *NumPy* [9]: Es la librería más utilizada en *Python*, que se basa en el tratamiento de objetos y datos numéricos. Su principal estructura es el *Ndarray*, que, a grandes rasgos, es un vector multidimensional.
- *Pandas* [11]: Está orientada al análisis de datos y proporciona estructuras de datos (como el *DataFrame*) y funciones para tratarlos. Un *DataFrame* se compone de un conjunto de *Series*, y se puede comparar con una tabla de datos.
- *Scikit-learn* [13]: Ofrece métricas y herramientas para evaluar los modelos de Aprendizaje Automático.
- *PyTorch* [12]: Está orientada al Aprendizaje Automático y facilita las labores de proceso al implementar lo que se conoce como *Tensor*, que es una estructura multidimensional cuya característica principal es que dentro de una misma dimensión los tensores no tienen por qué tener el mismo tamaño, permitiendo así, entradas de datos irregulares. Está basada en la biblioteca *Torch* [16] de *C*, que ya no se desarrolla, pero sigue siendo importante.

El resto del trabajo se estructura de la siguiente forma. En la Sección 2 se da un marco teórico donde se explican algunos de los conceptos usados en este trabajo, en la Sección 3 se analiza el conjunto de datos y se explican las transformaciones realizadas sobre el mismo, en la Sección 4 se realizan las primeras evaluaciones sobre el conjunto de datos, probando varios modelos y arquitecturas, en la Sección 5 se explican y se evalúan las aproximaciones Multi Tarea, y en la Sección 6 se dan las conclusiones sobre el trabajo realizado.

2. Marco teórico

Antes de comenzar a profundizar en el trabajo, es conveniente entender algunos de los conceptos que se tratan en él. En concreto, se van a explicar en qué consisten las tareas de clasificación y algunos conceptos sobre la Inteligencia Artificial y alguna de sus ramas. La Sección finaliza con la explicación de los métodos de evaluación más usados en este tipo de técnicas.

2.1. Tarea de clasificación

En este trabajo se van a analizar distintos tipos de datos, como datos estructurados e imágenes. La visión por computador es una disciplina que engloba diferentes técnicas y métodos que permiten analizar y extraer información de dichas imágenes. Entre las tareas de visión por computador clásicas destacan las tareas de clasificación, detección de objetos o la segmentación semántica o de instancias. Este trabajo se centra en la tarea de clasificación. Esta tarea consiste en dado un conjunto de datos y un conjunto de etiquetas, asignar a cada dato la etiqueta correcta. Este problema en concreto se centra en la clasificación binaria (ver Figura 1). En este tipo de clasificación hay únicamente dos etiquetas o clases, la clase positiva, con las instancias que estén etiquetadas con cáncer de mama, y la clase negativa, con aquellas etiquetadas sin cáncer de mama. Para resolver esta tarea a lo largo de los años se han usado diferentes técnicas de Inteligencia Artificial. Sin embargo, con el auge de las técnicas de Aprendizaje Profundo, estos problemas se están resolviendo con este tipo de modelos dando buen rendimiento.

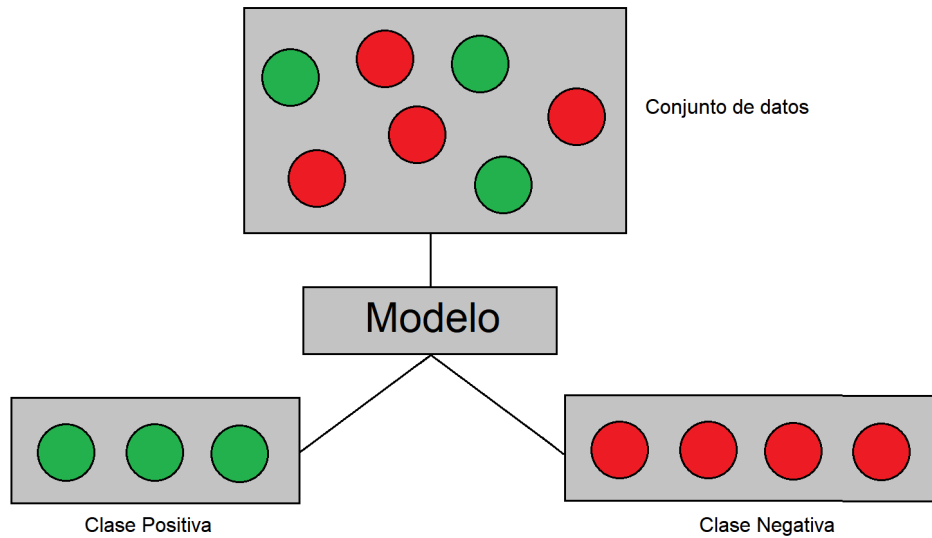


Figura 1: Ejemplo de clasificación en dos clases. Siendo la clase positiva aquella con las instancias etiquetadas con el color verde y la clase negativa aquella con las instancias etiquetadas con el color rojo.

2.2. Inteligencia Artificial, Aprendizaje Automático y Aprendizaje Profundo

La Inteligencia Artificial es un campo que busca reproducir aspectos del pensamiento humano mediante el uso de la tecnología. La idea principal es procesar información, aprender de ella y utilizar lo aprendido para realizar tareas costosas y repetitivas de una manera más cómoda y rápida [22].

Entre las diferentes técnicas de Inteligencia Artificial se puede encontrar el Aprendizaje Automático. Esta técnica está centrada en el desarrollo de sistemas y algoritmos que sean capaces de aprender y tomar decisiones sin ser programados de manera explícita para ello. Es decir, se programan para que aprendan a realizar la tarea, no para que la realicen directamente. En concreto, estas técnicas están basadas en el análisis de datos, la extracción de patrones y la toma de decisiones con una intervención mínima de los usuarios [19].

Dentro del Aprendizaje Automático se encuentran las técnicas de Aprendizaje Profundo. Estas técnicas se basan en Redes Neuronales profundas para aprender y extraer características comunes de los datos. Su nombre, profundo, viene debido a la cantidad de capas que tienen las Redes Neuronales. Está inspirado en la estructura y funcionamiento del cerebro

humano, en el que cada zona del cerebro se encarga de una tarea, tal y como cada capa dentro de la Red Neuronal se encarga de una tarea o una parte concreta [20].

La relación y la estructura de estos tres conceptos se puede ver en la Figura 2.

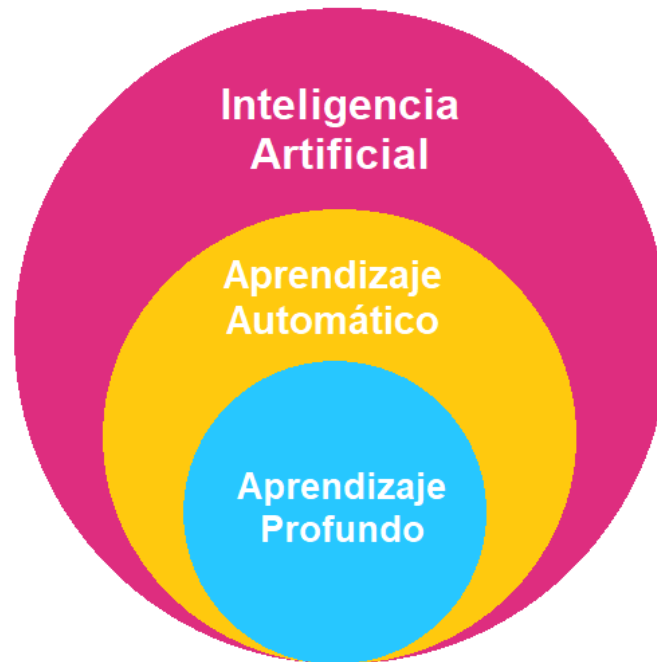


Figura 2: Estructura de la Inteligencia Artificial, el Aprendizaje Automático y el Aprendizaje Profundo.

2.3. Modelos y arquitecturas

En Aprendizaje Profundo, se le llama modelo a toda aquella arquitectura que cuenta con la capacidad de extraer información de los datos. Estos datos pueden ser tablas o imágenes. En el caso de las imágenes, los modelos más comunes en estas labores se basan en arquitecturas de *CNN* (Redes Neuronales Convolucionales), *RNN* (Redes Neuronales Recurrentes), *GAN* (Redes Adversarias Generativas) y Transformadores. De todas estas arquitecturas, las que realizan tareas de clasificación son las *CNN* y los Transformadores, por tanto, son las que se van a utilizar en este trabajo.

Las *CNN* son un tipo de Red Neuronal profunda diseñada para procesar datos con una estructura en forma de cuadrícula, por lo que funciona muy bien con imágenes y con audio. Su funcionamiento está basado en el concepto de convolución, que consiste en la extracción de características (como bordes, líneas, patrones...) mediante la aplicación de filtros sobre las

imágenes. De forma resumida, funciona reduciendo la dimensión de los datos para quedarse con las características importantes y poder clasificar mejor. Un ejemplo de su estructura se puede ver en la Figura 3.

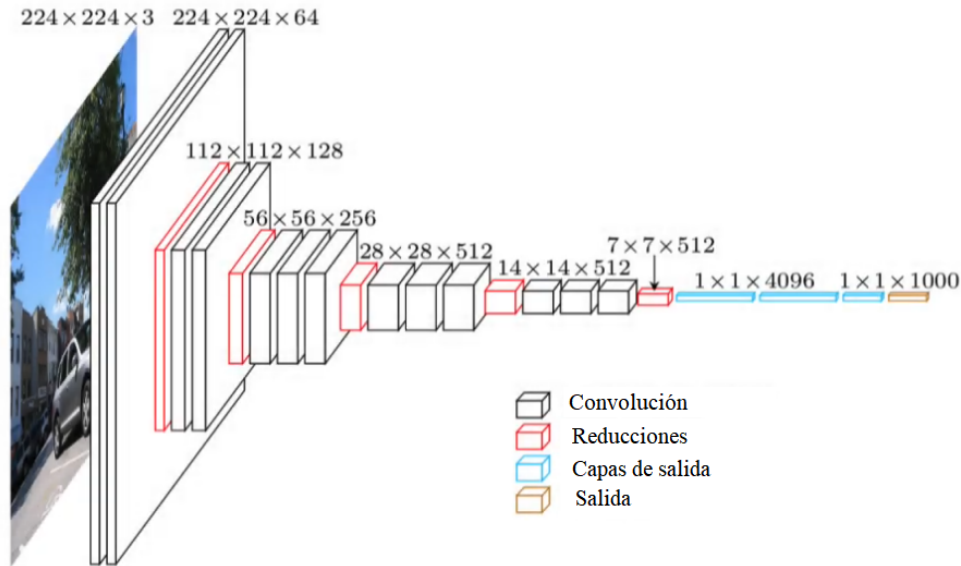


Figura 3: Visualización de una arquitectura de Red Neuronal Convolucional.

Los Transformadores son un tipo de Red Neuronal diseñado con dos componentes, codificadores y decodificadores. El codificador procesa la entrada y el decodificador genera la salida con los datos procesados. La característica principal que diferencia a los Transformadores del resto de Redes Neuronales es la capacidad de capturar información contextual en paralelo, en vez de hacerlo de manera secuencial. Esto los convierte en arquitecturas más eficientes en términos de tiempo de entrenamiento y procesado. Un ejemplo de su estructura se puede ver en la Figura 4.

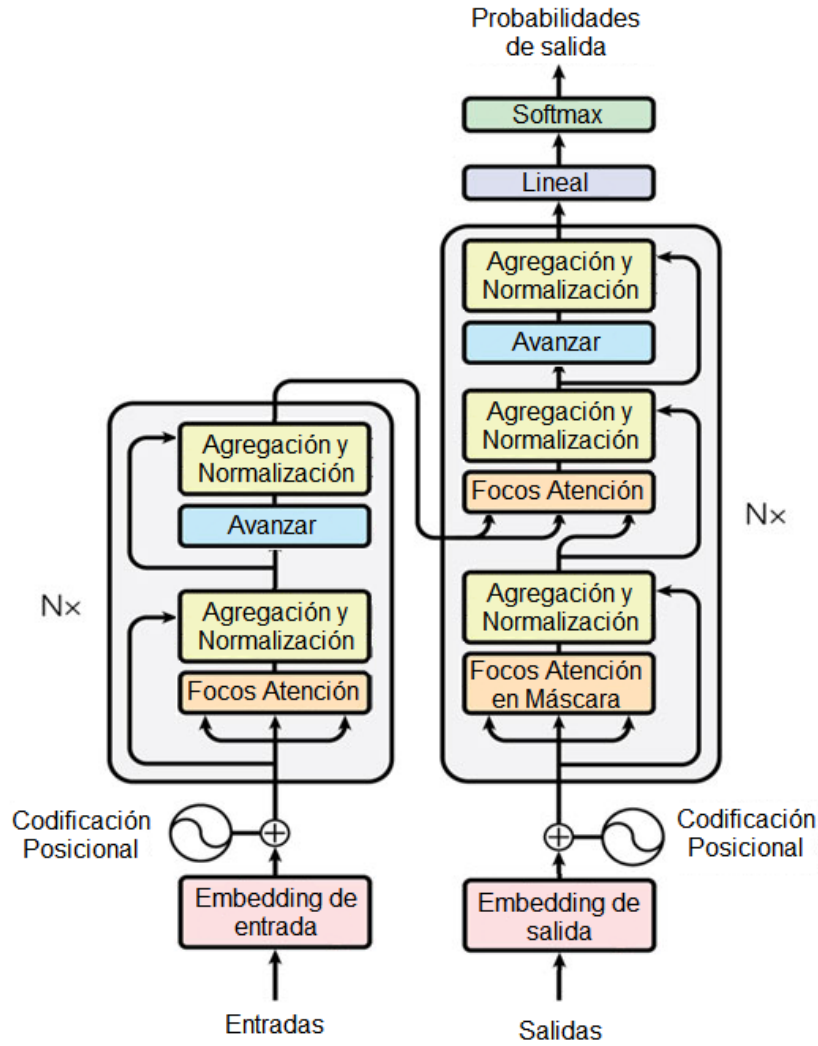


Figura 4: Visualización de una arquitectura de Transformador.

Todos estos modelos están orientados a trabajar con imágenes, sin embargo, como se ha mencionado anteriormente, en este trabajo se va a trabajar no únicamente con imágenes sino también con datos estructurados. La solución para este tipo de tareas es el aprendizaje Multi Tarea, que, pese a no ser la aproximación que más se está utilizando en los últimos años, va un paso más allá que los modelos vistos hasta este punto. Si bien, los modelos tradicionales tratan una entrada para dar una salida, es decir, dan una respuesta a un conjunto de datos, los modelos Multi Tarea realizan, tal y como indica su nombre, varias tareas sobre uno o varios conjuntos de datos. De esta forma, se puede extraer aún más información y al aprender características comunes a todas las tareas se puede dar una respuesta más precisa. También este tipo de modelos tienden a sobreajustar menos. Dado el problema que presenta este

trabajo, este tipo de modelos son los más adecuados, pues se busca tanto realizar varias predicciones distintas, como extraer información de varios tipos de datos a la vez. Estas dos aproximaciones a los modelos Multi Tarea, se explican en profundidad más adelante en el Apartado 5.

2.4. Métodos de evaluación

Para la evaluación de los modelos se utilizan diversas métricas. Cada métrica mide un aspecto en concreto del rendimiento del modelo, habiendo variedad para escoger la que recopile la información que interese en cada caso. Las más utilizadas son aquellas que recogen información sobre los resultados. En problemas de clasificación son las siguientes:

- Exactitud: En inglés se le llama *Accuracy*. Mide el porcentaje de instancias que se han etiquetado correctamente, asignándoles la etiqueta correspondiente a su clase. Se suele entender como el porcentaje de acierto del modelo.
- Precisión: En inglés se le llama *Precision*. Mide el porcentaje de instancias que se han etiquetado correctamente, pero teniendo en cuenta únicamente aquellas instancias clasificadas en la clase positiva. Es el porcentaje de acierto de las instancias que el modelo ha clasificado como positivas.
- Recuperación: En inglés se le llama *Recall* y también se la conoce como Exhaustividad. Mide el porcentaje de instancias que se han etiquetado correctamente, pero teniendo en cuenta únicamente aquellas instancias que pertenecen a la clase positiva. Es el porcentaje de acierto de las instancias que pertenecen a la clase positiva.
- Valor-F: En inglés se le llama *F-Score* o *F1-Score*. Se define como la media armónica sobre los valores de Precisión y Recuperación. Se puede ponderar y dar más importancia a una u otra métrica, pero en este trabajo, Precisión y Recuperación tienen el mismo peso a la hora de calcular el Valor-F. Esta métrica es la que mejor resume el rendimiento de un modelo, pues su valor no se ve afectado por el número de instancias en cada clase.

Todas estas métricas se pueden calcular a partir de la representación de los resultados del modelo, basado en la clasificación de las instancias. Esta representación se hace en forma de matriz, y se conoce como Matriz de Confusión, ver Figura 5.

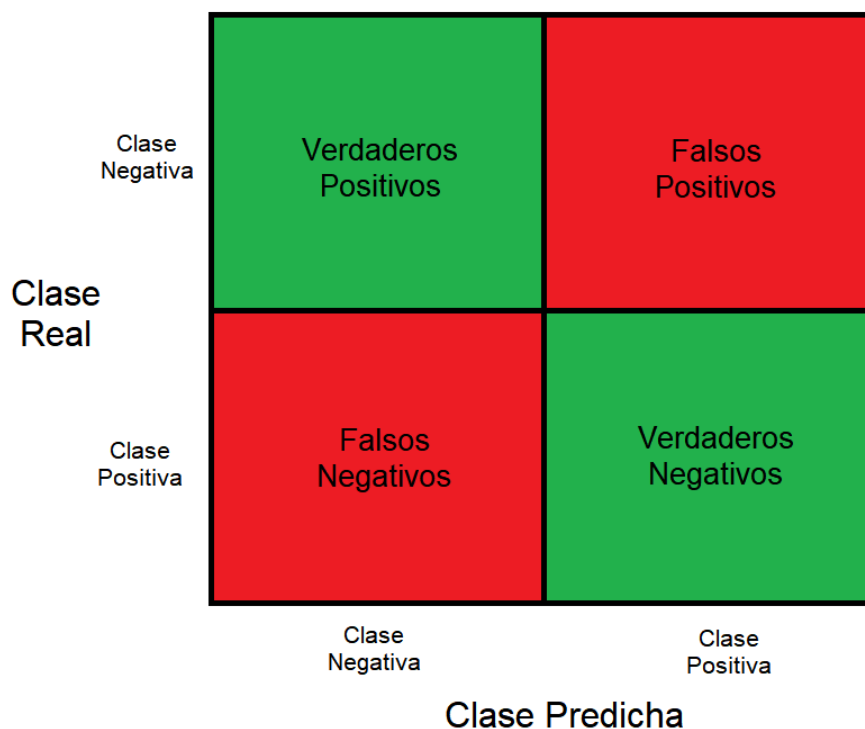


Figura 5: Visualización de una Matriz de Confusión.

Como se puede ver, la matriz tiene cuatro zonas bien diferenciadas. Estas son:

- Verdaderos Positivos (VP): Son aquellas instancias de la clase positiva que se han clasificado correctamente en la clase positiva.
- Falsos Positivos (FP): Son aquellas instancias de la clase negativa que se han clasificado incorrectamente en la clase positiva.
- Falsos Negativos (FN): Son aquellas instancias de la clase positiva que se han clasificado incorrectamente en la clase negativa.
- Verdaderos Negativos (VN): Son aquellas instancias de la clase negativa que se han clasificado correctamente en la clase negativa.

De estas zonas, las instancias acertadas están en los Verdaderos Positivos y en los Verdaderos Negativos, mientras que las instancias falladas están en los Falsos Positivos y los

Falsos Negativos. El modelo mejora al aumentar el valor en los VP y VN y al disminuir en FP y FN.

Las métricas que se han mencionado antes se pueden calcular a partir de la Matriz de Confusión (Figura 5):

$$Exactitud = \frac{VP + VN}{VP + VN + FP + FN}$$

$$Precision = \frac{VP}{VP + FP}$$

$$Recuperacion = \frac{VP}{VP + FN}$$

$$Valor - F = 2 * \frac{Precision * Recuperacion}{Precision + Recuperacion}$$

Lo que se debe buscar es aumentar el valor de todas estas métricas, pues cuánto más alto sea, mejor será el modelo, siendo 0 el peor valor posible, y 1 el mejor. Más adelante, se utiliza una métrica que mide el rendimiento del modelo, de manera inversa, siendo 1 el peor valor posible y 0 el mejor. Esta métrica es el error cuadrático medio (*RMSE*), que es una manera de medir cuánto se alejan las predicciones de los valores reales.

$$RMSE = \sqrt{(f - o)^2}$$

3. Análisis de los datos

En este apartado se explica el conjunto de datos que se utiliza en este trabajo. También se detallan los tratamientos y transformaciones realizadas sobre el conjunto, así como su limpieza.

3.1. Conjunto de datos

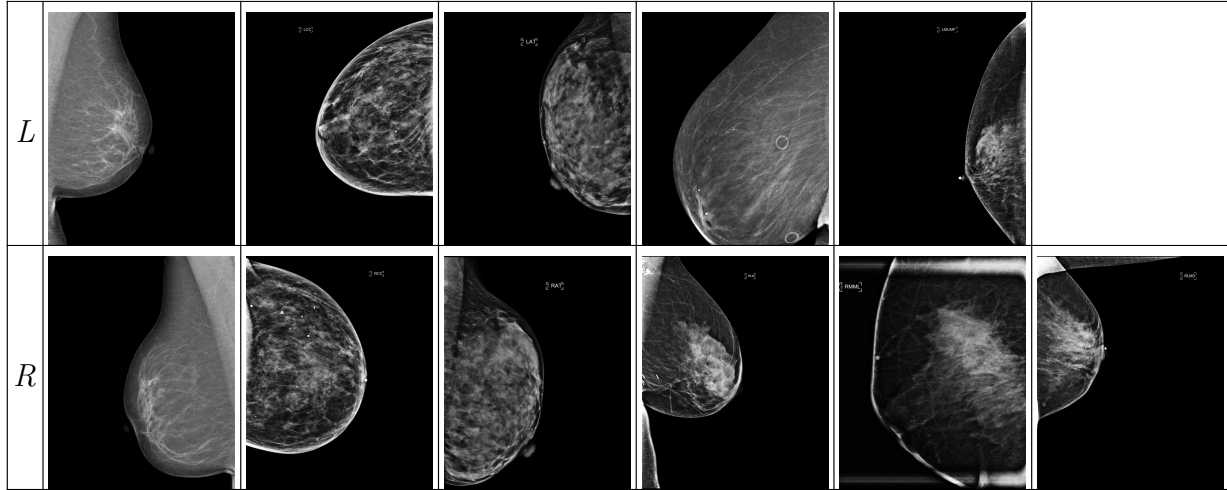
El conjunto de datos es el *RSNA Screening Mammography Breast Cancer Detection* [8], que cuenta con imágenes de mamografías y datos estructurados sobre cada paciente. Las imágenes de mamografías están organizadas en dos carpetas, una para los datos del conjunto de entrenamiento, *train_images*; y otra para los datos del conjunto de prueba, *test_images*. Además, las imágenes de entrenamiento están separadas por paciente, agrupando todas las imágenes de cada paciente en una misma carpeta. En total, en el conjunto de entrenamiento hay datos de 11.913 pacientes, es decir, 11.913 carpetas con imágenes. Por su parte, en la carpeta *test_images*, hay únicamente la carpeta de un paciente, con cuatro imágenes de prueba.

También, hay archivos en formato *.csv* (*Comma-Separated Values*) con los datos de los pacientes y de las imágenes, ver Cuadro 1. Estos archivos son *train.csv*, con el conjunto de Entrenamiento, *test.csv*, con el conjunto de Prueba y *sample_submission.csv*, con una muestra para la comprobación. Este último archivo se va a obviar, pues es un fichero propio de la competición de *Kaggle*. En estos archivos cada fila representa una imagen. Un análisis más en profundidad del conjunto de datos y su distribución se puede consultar en el Anexo A.1.

site_id	patient_id	image_id	laterality	view	age	cancer	biopsy	invasive	BIRADS	implant	density	machine_id	d	n	c
1	9973	1729524723	R	MLO	43.0	0	0	0	1.0	0	C	49			False

Cuadro 1: Ejemplo de instancia.

El conjunto total son 54.710 imágenes en formato *.DICOM*. Este formato (*Digital Imaging and Communication In Medicine*) es el estándar que se utiliza en medicina para la transmisión de imágenes y datos. En el caso de este conjunto, las imágenes son las mamografías y los datos asociados a ellas son el identificador del paciente, el tipo de imagen, el tamaño de la imagen, la fecha de la imagen y muchas otras especificaciones técnicas sobre la imagen. En el Cuadro 2 hay un ejemplo de cómo son las mamografías sobre las diferentes vistas desde las que son tomadas.



Cuadro 2: Mamografías de ejemplo, según sus vistas.

En el conjunto sólo hay una imagen con la vista *LMO*, y es del pecho derecho, por lo que no hay imagen en esta vista para la mama izquierda en el Cuadro 2. Se puede ver que son imágenes en escala de grises en donde el fondo aparece en un tono oscuro mientras que la mama se ve porque está resaltada en un tono más claro.

Las imágenes están estructuradas de forma que por cada paciente hay cuatro imágenes, dos por cada pecho (este es el estándar, pero hay algunos pacientes que tienen un número distinto de imágenes). Cada imagen tiene como nombre de archivo su identificador en la tabla de datos y están dentro de una carpeta con el nombre del identificador del paciente.

Como este trabajo no está orientado a participar en la competición de *Kaggle*, no se va a tomar en cuenta el archivo *sample_submission.csv*, y se va a obviar el archivo *test.csv*, pues sólo cuenta con 4 entradas, y prácticamente el total del conjunto de datos están en *train.csv* (quitando las 4 imágenes del conjunto de Prueba, éste tiene 54.706 imágenes, las de la carpeta *train_images*), y éste será el conjunto que se usará de aquí en adelante.

3.2. Reducción de volumen

El primer problema que plantea el conjunto de datos es su gran tamaño. En total pesa 314,72 GB y esto hace que trabajar con él sea inviable con las herramientas con las que se dispone. Todo este volumen viene por parte de las imágenes, que presentan dos características que las hacen especialmente pesadas. La primera es el propio formato, como se ha mencionado antes, el formato *.DICOM* adjunta a las imágenes cierta información, de la que realmente sólo interesa el identificador del paciente. Sin embargo, esta información realmente tampoco es necesaria, pues únicamente con el nombre de la imagen es suficiente para ubicarla dentro del *.csv*. Por lo tanto, no es necesario que la imagen esté en formato *.DICOM*, así que se

transforma a *.jpg*. Esta transformación reduce el tamaño total del conjunto de datos hasta los 112,49 GB.

En segundo lugar, las imágenes son en su mayoría de tamaño 2082x2776 o 4915x5355. Los modelos de aprendizaje profundo suelen utilizar tamaños de imagen más pequeños, por lo que el segundo paso es reducir la dimensión de las imágenes. Manteniendo las proporciones generales de las imágenes, se han reducido a un tamaño de 512x640 píxeles. Realizando estas transformaciones el tamaño total del conjunto es de 5,72 GB, lo que lo hace mucho más manejable.

3.3. Limpieza del conjunto de datos

Gran parte de los valores de la variable *'density'*, son valores faltantes (*NaN*), concretamente 25.236 instancias, que es un 46 % del total del conjunto. También hay unos pocos valores faltantes en la variable *'age'* (concretamente 37 instancias). Esta distribución se puede ver en la Figura 6.

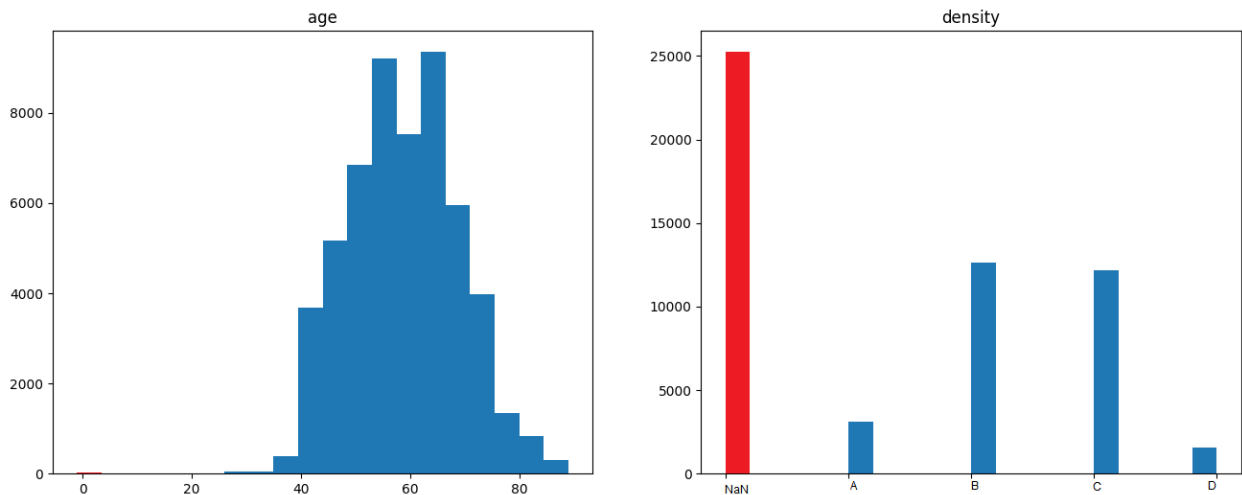


Figura 6: Visualización de la distribución de las variables *'age'* y *'density'*. Los valores faltantes se ven a la izquierda de cada gráfica, y están resaltados en color rojo. En el eje X, se ve el valor de las instancias, y en el eje y, la cuenta con las instancias con ese valor (o ese rango de valores en el caso de la edad).

Hay que tener en cuenta que este es un conjunto de datos real y en medicina los casos similares suelen tener características también parecidas. Es por eso por lo que los valores faltantes se pueden imputar mediante el uso de *KNN*, que clasifica las instancias en las categorías que tengan los casos que sean más similares. Sin embargo, al ser dos variables

importantes dentro del conjunto y, al contar con un gran número de instancias que tienen números faltantes, en vez de optar por realizar una imputación mediante *KNN*, se va a optar por no tener en cuenta aquellas instancias con valores faltantes en cualquiera de las dos variables. De esta manera se es más fiel al conjunto original y no se tienen datos sintéticos que puedan influir en los resultados. Haciendo esto, de las 57.710 instancias que se tenían en primer lugar, quedan 29.443. La distribución de la variable '*density*' queda de la siguiente manera:

- A: 3.105 (11 %).
- B: 12.642 (43 %).
- C: 12.155 (41 %).
- D: 1.531 (5 %).

Como se puede ver en el Anexo A.2, dentro de este nuevo conjunto sigue habiendo valores faltantes en la variable '*BIRADS*', concretamente 5.796 instancias. Sin embargo, esta variable únicamente está en el conjunto de entrenamiento, y no es necesaria para la predicción del cáncer, por lo que no es un problema que esta variable en concreto tenga instancias con valores faltantes. También en ese mismo anexo se puede ver que solamente quedan instancias del hospital con el identificador 1, por lo que la variable '*site_id*' se puede eliminar del conjunto, así como la variable '*implant*', que en el caso de este hospital no tiene un valor fiable.

3.4. División del conjunto de datos

Finalmente, para todas las pruebas con modelos que se van a realizar, se va a separar el conjunto en dos, uno para entrenar los modelos y otro para probar los resultados. El ratio que más se suele utilizar y que mejores resultados da es de un 80 % del total para el conjunto de entrenamiento y un 20 % para el conjunto de validación. Esta separación se hace mediante la función *RandomSplitter*, que separa las instancias de manera aleatoria. Esta será la separación de aquí en adelante para los modelos de imagen.

Para los modelos de datos, se va a utilizar una separación diferente. Un 70 % para el conjunto de entrenamiento, un 10 % para el conjunto de validación y un 20 % para el conjunto de prueba, mediante la función *train_test_split*. En este caso se evaluará el mejor modelo y los mejores parámetros con el conjunto de validación y, una vez hecho esto, reentrenar el modelo juntando el conjunto de entrenamiento y validación para dar los resultados finales sobre el conjunto de prueba. Esta división en tres partes sirve para ajustar mejor los parámetros del modelo.

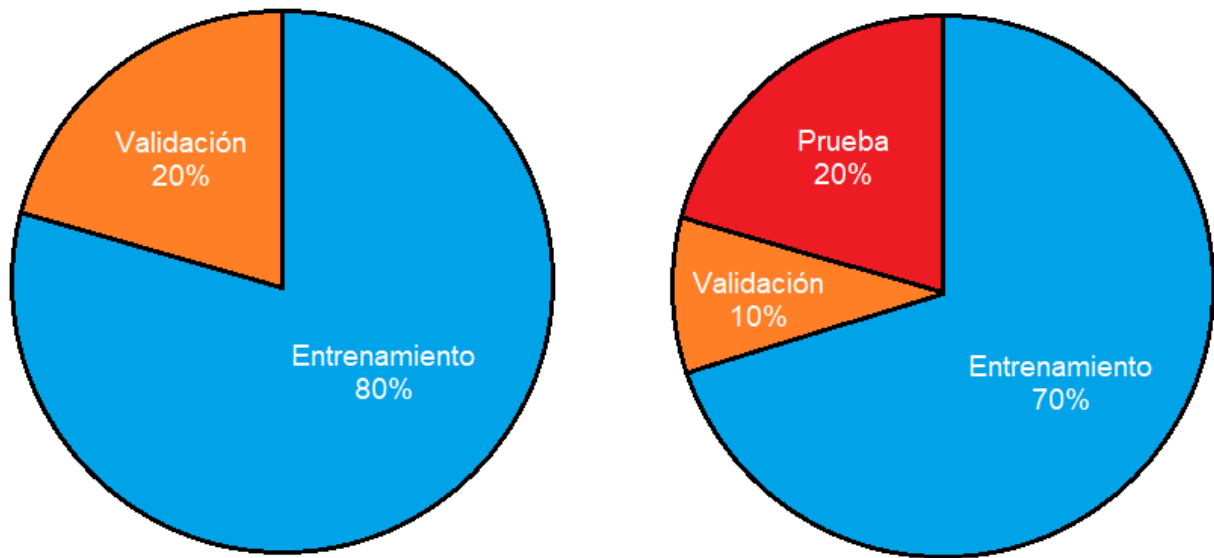


Figura 7: Visualización de la separación de los conjuntos en partes mediante una gráfica de sectores. Representando al conjunto de entrenamiento en azul, el conjunto de validación en naranja y el conjunto de prueba en rojo.

4. Evaluación inicial del problema

La idea inicial es entrenar dos modelos distintos, uno para obtener información de las mamografías y otro para predecir el cáncer con los datos asociados a las mismas de los *.csv*. Una vez se tengan ambos modelos, se buscará una aproximación mediante el uso de una arquitectura Multi Tarea, y se comprobará si los resultados mejoran o empeoran al usar ese tipo de modelo.

4.1. Modelos para imagen. Parte 1

Para el análisis de este problema se ha decidido estudiar el rendimiento de algunas de las arquitecturas más usadas en la literatura. En concreto, se han seleccionado tres arquitecturas basadas en *CNN* y 2 basadas en Transformadores:

- *EficientNet B3* [5], que es un modelo basado en arquitectura de Red Neuronal.
- *HrNet w44* [18], que es un modelo basado en arquitectura de Red Neuronal.
- *ResNet 50* [4], que es un modelo basado en arquitectura de Red Neuronal.
- *ConvNext* [6], que es un modelo basado en arquitectura Transformer.
- *DeiT3 base patch 16 384* [17], que es un modelo basado en arquitectura Transformer.

Para la evaluación de los distintos modelos sobre las imágenes, se ha probado con el conjunto tras aplicarle la limpieza y los modelos mencionados, modificando el factor de aprendizaje, los callbacks a la hora de entrenar y el número de épocas que se ejecutan. La comparación de resultados se realiza mediante tablas, en las que se muestran los mejores valores de cada prueba junto con los minutos tardados en llegar a ella y las épocas transcurridas desde el inicio. También, los resultados se evalúan respecto a la variable ‘*cancer*’, que es la variable objetivo del conjunto de datos. Las métricas usadas son las más comunes: Exactitud, Precisión, Recuperación y Valor-F, siendo esta última la más importante debido al tipo de problema al que se enfrenta este trabajo, pues es la más usada en el ámbito del diagnóstico médico y es peligroso fallar tanto en casos positivos como negativos.

Para realizar el entrenamiento de los modelos se ha usado *Google Colab*, que cuenta con un procesador *Intel Xeon CPU* con una frecuencia de 2.20 GHz, 13 GB de memoria RAM, un acelerador *Tesla K80* y 12 GB GDDR5 de VRAM. Sobre el conjunto de datos se realiza una separación del 80 % del conjunto para realizar el entrenamiento (43.768 imágenes), y el 20 % restante para probar los modelos (10.942 imágenes). Se entrena con el callback de *Early Stopping* sobre la métrica de Valor-F con una paciencia de 3 épocas. Esto quiere decir que si

el modelo no mejora sus resultados en la métrica Valor-F durante 3 épocas consecutivas, se detiene el entrenamiento. Esto previene el sobreajuste y optimiza el tiempo de entrenamiento. El ratio de aprendizaje, se ha ajustado a cada modelo para dar los mejores resultados usando la función *lr_find()* de *FastAI* [2], con unos valores comprendidos entre 1e-2 y 1e-3. Al ser un conjunto de datos con un gran número de imágenes, el número máximo de épocas que se va a entrenar el modelo son 12, aunque lo más probable es que al aplicar *Early Stopping* se detenga el entrenamiento antes de llegar a esa época. Estos resultados se pueden ver en el Cuadro 3.

	<i>EfficientNet</i>	<i>HrNet</i>	<i>ResNet</i>	<i>ConvNext</i>	<i>Deit</i>
Exactitud	0,9757	0,9757	0,9757	0,9757	0,9793
Precisión	0	0	0	0	0
Recuperación	0	0	0	0	0
Valor-F	0	0	0	0	0
Tiempo (Minutos)	5	11	5	7	15
Época	1	1	1	1	1

Cuadro 3: Resultados de las 5 arquitecturas sobre el conjunto de imágenes completo.

En el Cuadro 3, todos los modelos han obtenido unos resultados en la métrica Exactitud muy altos. Sin embargo, como se puede ver en las demás métricas, no es un buen resultado. De aquí se puede extraer que los modelos están clasificando todas las instancias como clase negativa, que es la clase mayoritaria, como se puede ver en el Anexo A.2. Es por eso por lo que es necesario realizar un balanceo de los datos. Esta idea queda reforzada al consultar la Matriz de Confusión (Figura 8).

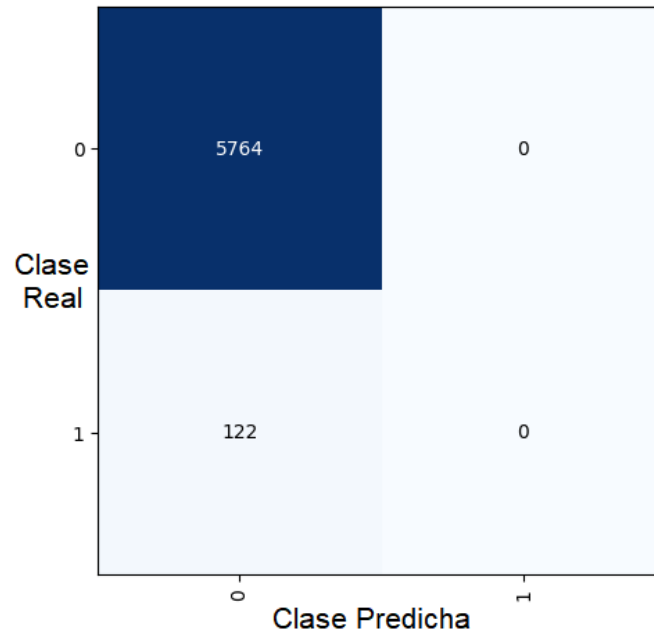


Figura 8: Visualización de la matriz de confusión sobre el conjunto completo (concretamente la del modelo *Deit*).

En la Figura 8 se puede observar la Matriz de Confusión, que indica cómo se han realizado las predicciones. En la diagonal principal, se encuentran los aciertos del modelo, mientras que, en la diagonal inversa, los fallos. En esta Figura se comprueba que no se está realizando ninguna predicción sobre la clase positiva. Podría ser que llame la atención que en la matriz hay 5.886 instancias en vez de las 29.443 que hay en total, y esto es debido a que tanto las métricas de las tablas como la matriz de confusión se calcula sobre el conjunto de validación.

4.2. Balanceo de datos

Como se puede ver en el Anexo A.2, para la variable objetivo del conjunto de datos, que es la variable *'cancer'*, no hay una distribución equilibrada (solamente el 2 % de las imágenes está catalogada como caso positivo en cáncer). Esto indica que es necesario balancear los datos para igualar el número de instancias en ambas clases.

Para balancear los datos las dos técnicas más extendidas son el *oversampling* y el *undersampling*. La primera consiste en aumentar de manera artificial el conjunto de la clase minoritaria para que haya un número parecido de instancias en ambas clases. La segunda, consiste en disminuir el tamaño de la clase mayoritaria, normalmente se hace mediante la selección aleatoria de instancias.

Hay dos opciones sobre cómo realizar este balanceo, ambas siguiendo la técnica de *undersampling*, porque, al ser un tema tan específico y en el que el fallo es tan castigado, crear datos sintéticos puede ser peligroso, por lo que es mejor usar los datos reales que ya se tienen. Además, esta forma de proceder sigue la misma línea que en la limpieza de datos (Apartado 3.3). Las dos opciones son balancear por pacientes o por imágenes. Esto se explica a continuación.

4.2.1. Balanceo por pacientes

En el conjunto de datos hay 252 pacientes que han dado positivo en alguna de sus imágenes. Aplicando esta técnica de balanceo, el resultado final es un conjunto de datos reducido, con 504 pacientes.

El conjunto de las imágenes queda reducido también a 2.597 imágenes, de las cuales 664 (25,6 %) tienen cáncer y 1.933 (74,4 %) no. Esta opción pese a ser una aproximación más realista, sigue estando desbalanceada, ya que los pacientes que han dado negativo tienen todas sus fotos con resultado negativo, mientras que los pacientes positivos tienen la mitad de sus fotos negativas y la mitad positivas (ya que lo normal es presentar cáncer de mama en un pecho y no en los dos).

4.2.2. Balanceo por imágenes

En el conjunto de datos hay 664 imágenes con cáncer. Tras aplicar el balanceo, quedan un total de 1.320 imágenes, las 664 positivas y 656 negativas. En este balanceo los pacientes con imágenes negativas tendrán generalmente una única imagen, mientras que aquellos con cáncer tendrán dos (las dos vistas estándar por cada pecho). Esto se aleja del caso real, pero se acerca más a los conjuntos de entrenamiento de modelos de Aprendizaje Profundo, que cuentan con conjuntos que presentan balanceos cercanos al 50 % porque son los que mejores resultados dan. Así que este es el conjunto que se va a utilizar de aquí en adelante.

4.3. Modelos para imagen. Parte 2

Con el conjunto de datos ya balanceado, se va a volver a evaluar de la misma manera que en el Apartado 4.1 pero usando este conjunto ya balanceado. Sobre este conjunto se realiza una separación que sigue siendo del 80 % del conjunto para realizar el entrenamiento (1.056 imágenes), y el 20 % restante para probar los modelos (264 imágenes). Como el volumen de datos es bastante inferior al original, el entrenamiento de los modelos es mucho más rápido,

por lo que se puede entrenar durante más épocas. Se va a evaluar sobre 20 épocas en este caso, aunque es poco probable que se llegue a una época cercana, pues se está utilizando el callback de *Early Stopping*. Estos resultados se encuentran en el Cuadro 4.

	<i>EfficientNet</i>	<i>HrNet</i>	<i>ResNet</i>	<i>ConvNext</i>	<i>Deit</i>
Exactitud	0,5038	0,5606	0,5455	0,5985	0,6439
Precisión	0,5556	0,5733	0,5422	0,7324	0,6619
Recuperación	0,2878	0,6475	0,8777	0,3741	0,6619
Valor-F	0,3792	0,6081	0,6703	0,4952	0,6619
Tiempo (Minutos)	0,5	4	0,5	4	1,5
Época	1	2	1	3	2

Cuadro 4: Resultados de las 5 arquitecturas sobre el conjunto de imágenes balanceado, con el mejor resultado del Valor-F resaltado en **negrita**.

En el Cuadro 4 se ha conseguido un Valor-F de 0,6703, mejor aún que en el Cuadro 3, cuyo mejor valor era 0,000. De esta manera se han conseguido los mejores resultados con el modelo *ResNet* y, usando el callback de *Early Stopping*, se optimiza el tiempo de entrenamiento y además se consigue evitar el sobreajuste. Como comprobación, se va a volver a consultar los resultados de la Matriz de Confusión (Figura 9).

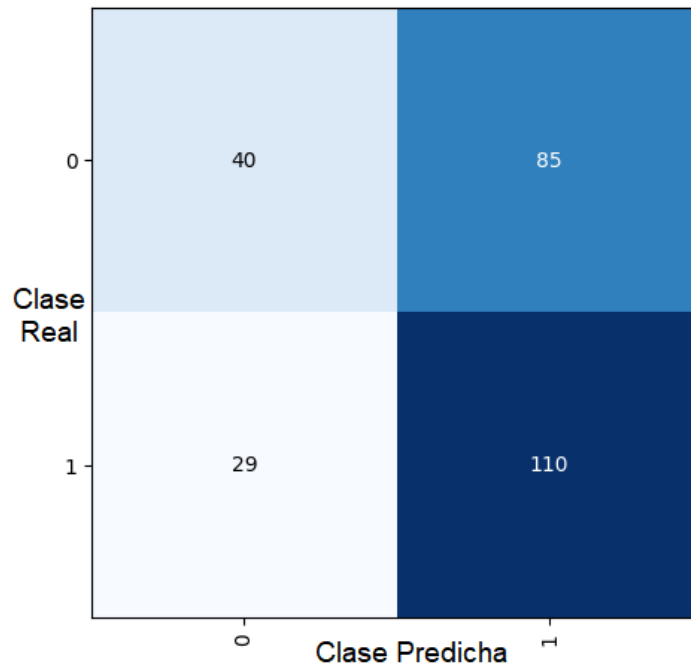


Figura 9: Visualización de la matriz de confusión sobre el conjunto balanceado (concretamente la del modelo *ResNet*).

Como se puede ver en la Figura 9, ahora sí que se están prediciendo los casos en la clase positiva, de hecho, se ve que muchos de los fallos que tiene son por diagnosticar cáncer a instancias que no lo tienen.

4.4. Modelos para datos. Parte 1

Siguiendo con el conjunto balanceado, se utilizan los datos asociados a las imágenes y se preparan para entrenar los modelos, que necesitan dos conjuntos, X (con las variables con las que calcular la variable objetivo) e y (con la variable objetivo). Teniendo en cuenta que en el conjunto de datos hay variables que se aportan una vez se ha detectado el cáncer, no serán tomadas en cuenta. Los conjuntos quedan de la siguiente manera:

- X :
 - *'patient_id'*.
 - *'image_id'*.
 - *'laterality'*.
 - *'view'*.
 - *'age'*.
 - *'density'*.
 - *'machine_id'*.
- y :
 - *'cancer'*.

Y se distribuye así:

- 70 % para el conjunto de entrenamiento (924 imágenes).
- 10 % para el conjunto de validación (132 imágenes).
- 20 % para el conjunto de prueba (264 imágenes).

Las variables desechadas en este conjunto son *'invasive'*, *'biopsy'*, *'BIRADS'* y *'difficult_negative_case'*, que son aquellas que en primera instancia sólo aparecían en el conjunto de Entrenamiento. También para poder procesar mejor el conjunto de datos, a las variables categóricas (*'laterality'*, *'view'* y *'density'*) se les aplica codificación ordinal, que le asigna un número a cada clase siguiendo el orden de aparición.

Para evaluar los datos, los modelos seleccionados son los siguientes:

- *Árbol C4.5.*
- *Árbol CART.*
- *Random Forest.*
- *Regresión Logística.*
- *Red Neuronal.*
- *SVM.*

Los resultados de estos modelos se pueden observar en el Cuadro 5 (Los detalles sobre los parámetros de los modelos están en el Anexo C):

	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,6818	0,6742	0,7197	0,4545	0,4545	0,5455
Valor-F	0,6866	0,6815	0,7218	0,0000	0,0000	0,7059
Falsos positivos	0,1212	0,1288	0,0985	0,0000	0,0000	0,4546
Falsos negativos	0,1970	0,1970	0,1818	0,5455	0,5455	0,0000

Cuadro 5: Resultados de las 6 arquitecturas sobre el conjunto de validación con todas las variables, con el mejor resultado del Valor-F resaltado en **negrita**.

En el Cuadro 5 el mejor modelo ha sido el *Random Forest*, con un Valor-F de 0,7218 pero se puede ver que en otros modelos hay clases que no obtienen representación (los modelos de *Regresión Logística* y *Red Neuronal*). Esto es porque en estas clases no se están realizando predicciones sobre la clase positiva, y puede ser que lo que provoque este comportamiento es que los datos no están normalizados y se esté dando más peso a las variables con valores más altos. Si se normalizan los datos y se repite la evaluación de los modelos se obtienen los resultados del Cuadro 6

	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,5455	0,6591	0,6515	0,6591	0,6818	0,6439
Valor-F	0,5455	0,6667	0,6349	0,6763	0,7083	0,6667
Falsos positivos	0,1818	0,1364	0,1061	0,1515	0,1591	0,1667
Falsos negativos	0,2727	0,2046	0,2424	0,1894	0,1591	0,1894

Cuadro 6: Resultados de las 6 arquitecturas sobre el conjunto de validación con todas las variables aplicando normalización, con el mejor resultado del Valor-F resaltado en **negrita**.

En el Cuadro 6 el modelo *Red Neuronal* ha dado los mejores resultados, con un Valor-F de 0,7083. Si bien, los resultados han mejorado respecto a los modelos que no clasificaban bien en el Cuadro 5, pero en el resto de los modelos, los resultados han empeorado. El mejor resultado

en conjunto lo ha dado *Random Forest* sin normalizar, con un Valor-F de 0,7218. Esto puede ser porque las variables con los valores más elevados sean las que más importancia tengan a la hora de calcular el resultado. De ser cierto, habría que omitir mediante selección de características aquellas variables que generen ruido y que no aporten información relevante.

4.5. Selección de características

En el conjunto de datos hay variables con las que no tiene mucho sentido entrenar el modelo, como pueden ser los identificadores. Con el objetivo de optimizar los resultados de la búsqueda del mejor conjunto de datos para el modelo, se ha creado una función para calcular todas las posibles combinaciones de variables. Esto, se puede resumir como probar todas las posibles combinaciones de las columnas del conjunto de datos y ver qué combinación es la que mejores resultados da. Entre las posibles combinaciones está el conjunto vacío, pero por razones obvias, este conjunto no será utilizado para evaluar.

Es también importante mencionar, que los identificadores no van a ser tomados como variables a la hora de entrenar, ya que, aunque hacer esto mejora bastante los resultados, no tiene sentido, pues no tiene por qué seguir un mismo patrón a la hora de añadir nuevos datos, o lo que es lo mismo, pone en grave peligro la generalización del modelo y de esta manera se evita el sobreajuste.

Antes de realizar ninguna combinación, se evalúan los resultados de los modelos sobre todo el conjunto, pero sin tener en cuenta las variables que representan algún tipo de identificador (omitiendo las variables *'patient_id'*, *'image_id'* y *'machine_id'*), como se puede ver en el Cuadro 7. El objetivo de esta evaluación es el de buscar el mejor modelo para este conjunto de datos, para probar después las distintas combinaciones de variables con ese modelo.

	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,6515	0,6742	0,5909	0,6288	0,6136	0,6667
Valor-F	0,6167	0,6667	0,6351	0,6260	0,6623	0,6857
Falsos positivos	0,0833	0,1061	0,2197	0,1364	0,2197	0,1515
Falsos negativos	0,2652	0,2197	0,1894	0,2349	0,1667	0,1818

Cuadro 7: Resultados de las 6 arquitecturas sobre el conjunto de validación con todas las variables salvo los identificadores, con el mejor resultado del Valor-F resaltado en **negrita**.

En el Cuadro 7 el modelo que mejor resultado ha dado con el conjunto de datos ha sido el *SVM*, por lo que las combinaciones y sus resultados se calcularán utilizando este modelo como base (Cuadro 8).

Variables	Exactitud	Valor-F	Falsos Positivos	Falsos Negativos
<i>'lateralidad'</i>	0,5207	0,5207	0,2490	0,2301
<i>'view'</i>	0,5170	0,4754	0,2113	0,2716
<i>'age'</i>	0,6377	0,6308	0,1811	0,1811
<i>'density'</i>	0,5660	0,5382	0,1962	0,2377
<i>'lateralidad', 'view'</i>	0,5283	0,5020	0,2188	0,2528
<i>'lateralidad', 'age'</i>	0,5925	0,6250	0,2566	0,1509
<i>'lateralidad', 'density'</i>	0,5585	0,6139	0,3018	0,1396
<i>'view', 'age'</i>	0,6075	0,6119	0,2113	0,1811
<i>'view', 'density'</i>	0,5283	0,6057	0,3433	0,1283
<i>'age', 'density'</i>	0,6717	0,6587	0,1547	0,1735
<i>'lateralidad', 'view', 'age'</i>	0,6000	0,5985	0,2075	0,1924
<i>'lateralidad', 'view', 'density'</i>	0,5396	0,5515	0,2528	0,2075
<i>'lateralidad', 'age', 'density'</i>	0,6264	0,6207	0,1886	0,1849
<i>'view', 'age', 'density'</i>	0,6415	0,6468	0,1962	0,1622
<i>'lateralidad', 'view', 'age', 'density'</i>	0,6113	0,6113	0,2037	0,1849

Cuadro 8: Resultados de la arquitectura *SVM* sobre el conjunto de datos sin las variables con identificadores y las posibles combinaciones de variables, con el mejor resultado del Valor-F resaltado en **negrita**.

En el Cuadro 8 se ven algunos resultados interesantes, al estar sin normalizar, la variable *'age'* tiene más peso que las demás a la hora de emitir un resultado pues es la que cuenta con los valores más altos. Si se normalizan las variables para dar a cada una de ellas el mismo grado de importancia, los resultados se encuentran en el Cuadro 9.

Variables	Exactitud	Valor-F	Falsos Positivos	Falsos Negativos
'lateralidad'	0,5207	0,5207	0,2490	0,2301
'view'	0,5132	0,4735	0,2150	0,2716
'age'	0,6453	0,6299	0,1660	0,1886
'density'	0,5660	0,5382	0,1962	0,2377
'lateralidad', 'view'	0,5245	0,5000	0,2226	0,2528
'lateralidad', 'age'	0,6113	0,6053	0,1962	0,1924
'lateralidad', 'density'	0,5472	0,6129	0,3207	0,1320
'view', 'age'	0,6453	0,6328	0,1698	0,1849
'view', 'density'	0,5321	0,6076	0,3396	0,1283
'age', 'density'	0,6377	0,6220	0,1698	0,1924
'lateralidad', 'view', 'age'	0,6189	0,6189	0,2000	0,1811
'lateralidad', 'view', 'density'	0,5396	0,5579	0,2603	0,2000
'lateralidad', 'age', 'density'	0,6113	0,6113	0,2037	0,1849
'view', 'age', 'density'	0,6377	0,6250	0,1735	0,1886
'lateralidad', 'view', 'age', 'density'	0,6075	0,6090	0,2075	0,1849

Cuadro 9: Resultados de la arquitectura *SVM* sobre el conjunto de datos normalizando sin las variables con identificadores y las posibles combinaciones de variables, con el mejor resultado del Valor-F resaltado en **negrita**.

En el Cuadro 9 se puede observar que el rendimiento baja respecto al conjunto sin normalizar (Cuadro 8), ya que el valor de la edad es la característica más importante del conjunto, y al normalizarlo, pierde fuerza en el resultado final. Si se comparan las filas normalizadas y sin normalizar, se puede ver que aquellas sin la variable 'age' no presentan cambios, mientras que las que sí, generalmente pierden en rendimiento.

Como se puede ver, la mejor combinación posible de variables es aquella que contiene únicamente las variables 'age' y 'density'.

4.6. Modelos para datos. Parte 2

Se va a volver a evaluar reduciendo el número de variables del conjunto de datos, tal y como se explica en el Apartado 4.5. Los resultados en el conjunto de validación se encuentran en el Cuadro 10.

	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,6591	0,6591	0,6818	0,6439	0,6288	0,6591
Valor-F	0,6565	0,6565	0,6912	0,6466	0,6016	0,6512
Falsos positivos	0,1212	0,1212	0,1288	0,1364	0,1061	0,1136
Falsos negativos	0,2197	0,2197	0,1894	0,2197	0,2652	0,2273

Cuadro 10: Resultados de las 6 arquitecturas sobre el conjunto de validación con las mejores variables, con el mejor resultado del Valor-F resaltado en **negrita**.

En el Cuadro 10 el modelo que ha dado mejores resultados es el *Random Forest*, con un Valor-F de 0,6912. Sin embargo, el resultado final depende del conjunto de prueba. Para ello, se añade al conjunto de entrenamiento el de validación, se reentrenan los modelos con este conjunto, utilizando los mejores parámetros encontrados en la anterior prueba, y se evalúan sobre el de prueba. Los resultados finales se encuentran en el Cuadro 11.

	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,6604	0,6604	0,6566	0,6415	0,6453	0,6717
Valor-F	0,6617	0,6642	0,6642	0,6360	0,6493	0,6742
Falsos positivos	0,1811	0,1849	0,1925	0,1811	0,1925	0,1774
Falsos negativos	0,1585	0,1547	0,1509	0,1774	0,1623	0,1509

Cuadro 11: Resultados de las 6 arquitecturas sobre el conjunto de prueba con las mejores variables, con el mejor resultado del Valor-F resaltado en **negrita**.

En el Cuadro 11 el mejor modelo es el *SVM*, que obtiene los mejores resultados en todas las métricas. Concretamente en Valor-F ha obtenido un valor de 0,6742. Este es el modelo que se utilizará de aquí en adelante para procesar los datos.

5. Aproximaciones Multi Tarea

Hasta ahora se han estudiado los datos en forma de imagen y los datos estructurados por separado. En esta Sección se van a analizar cómo el uso de más tipos de datos afecta al rendimiento de los modelos. Es por ello por lo que se va a utilizar un modelo Multi Tarea, de forma que se extraiga más información de todos los datos a la vez y se obtenga un modelo más robusto. Estos modelos buscan resolver varias tareas relacionadas a la vez, en vez de usar un modelo para cada tarea individual. Este tipo de modelos generalizan mejor y, al aprender las características comunes a las tareas y compartir información entre esas características, mejora los resultados frente a los modelos individuales. Hay dos aproximaciones para crear un modelo Multi Tarea.

5.1. Primera aproximación

La primera aproximación se basa en el fusionado de diferentes tipos de datos. En concreto en esta tarea, se quiere extraer información de las imágenes de mamografías y de los datos estructurados para predecir si un paciente va a tener cáncer o no. Para realizar esta fusión existen tres posibilidades: *Early Fusion*, *Middle Fusion* y *Late Fusion*.

La técnica de *Early Fusion* consiste en fusionar los datos antes de extraer las características, y, posteriormente, realizar la clasificación sobre las características extraídas, como se puede ver en la Figura 10.

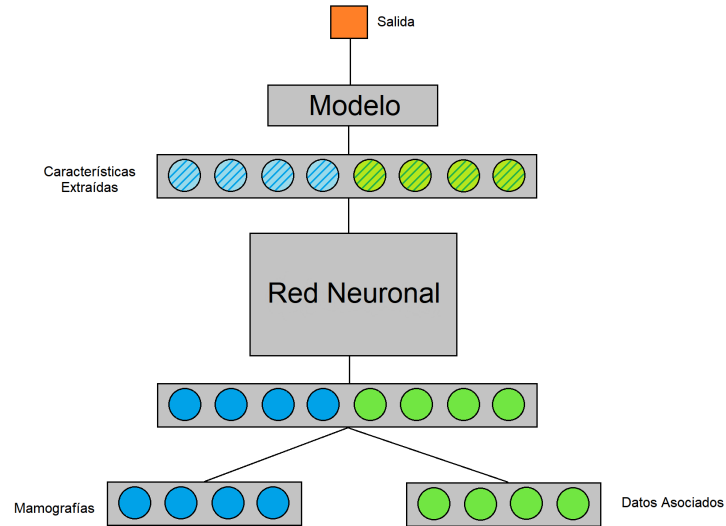


Figura 10: Aproximación 1 para el modelo Multi Tarea, utilizando la variante de *Early Fusion*.

La técnica de *Middle Fusion* consiste en extraer las características de las imágenes antes de fusionar los datos. Posteriormente, realizar el conjunto de datos fusionado, como se puede ver en la Figura 11.

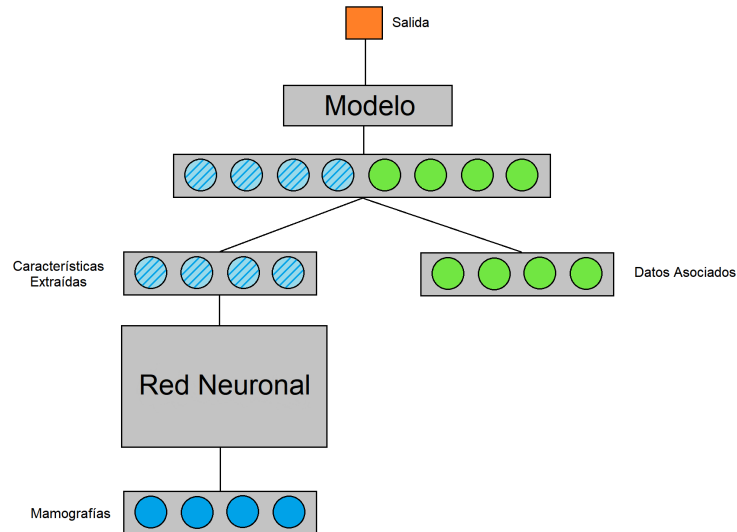


Figura 11: Aproximación 1 para el modelo Multi Tarea, utilizando la variante de *Middle Fusion*.

La técnica de *Late Fusion* consiste en extraer las características de imágenes y datos

estructurados por separado. Posteriormente, se fusionan las salidas de ambos modelos y se realiza la clasificación con las salidas de ambos modelos, tal y como se puede ver en la Figura 12.

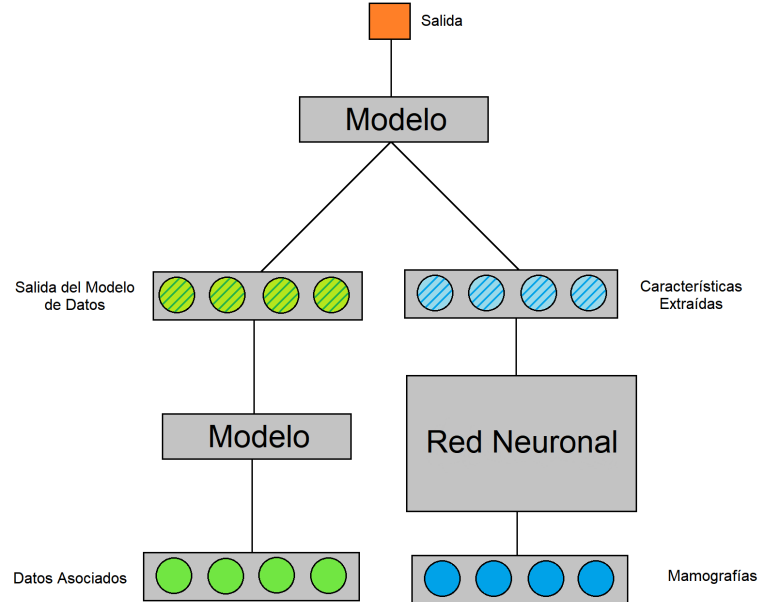


Figura 12: Aproximación 1 para el modelo Multi Tarea, utilizando la variante de *Late Fusion*.

De estas 3 variaciones, se ha decidido utilizar la de *Middle Fusion* (Figura 11), que es la que mejor se adapta a este problema. En este caso sería extraer mediante un extractor de características, las características de las mamografías, para posteriormente fusionar estas características con los datos de la edad y la densidad. Una vez fusionado el conjunto de datos, se utiliza para entrenar un modelo de clasificación como los vistos en el Apartado 4.4. Para extraer las características se van a usar los modelos entrenados en el Apartado 4.1, a las que hay que quitarles un número determinado de capas en la última instancia del modelo para recoger las características de las imágenes en vez de la predicción sobre la variable ‘cancer’.

- *EficientNet B3*: se quitan las cuatro últimas capas.
- *HrNet w44*: se quitan las cuatro últimas capas.
- *ResNet 50*: se quitan las cuatro últimas capas.
- *ConvNext*: se quitan las cuatro últimas capas.
- *Deit3 base patch 16 384*: se quitan las dos últimas capas.

Tras adaptar los modelos, se extraen las características de los conjuntos de datos (80 % conjunto de entrenamiento y 20 % para el de validación) y se crean los *DataFrames* juntando las variables ‘age’ y ‘density’ con las características extraídas. Esto se hace mediante la función *creaDF()*, ver Figura 13.

```

1 def creaDF(imagenes, caracteristicas, df):
2     dfb = df[['age', 'cancer', 'density']].copy() # Se seleccionan las
    variables objetivo
3     lista = []
4     for i in range(len(caracteristicas)): # Por cada vector de
        caracteristicas
5         img = imagenes[i]
6         serie = dfb.iloc[img]
7         car = caracteristicas[i].cpu().numpy() # Se transforma el vector
        de caracteristicas de tensor a ndarray
8         for j in range(len(car)):
9             serie = pd.concat([serie, pd.Series({j: car[j]})]) # Por cada
        caracteristica, se agrega una columna a la serie
10        lista.append(serie)
11    return pd.DataFrame(lista) # Se crea una DataFrame con todas las
    Series

```

Figura 13: Código de la función *creaDF*, que realiza la fusión de los datos.

A esta función se le pasan la lista de imágenes del conjunto en concreto junto a los vectores de características asociados a ellas. También se le pasa como argumento el *DataFrame* con los datos para poder extraer la edad y la densidad de cada imagen. Cada vector tiene 512 características. El *DataFrame* de salida se separa en *X*, con todas las variables salvo la variable objetivo, e *y*, con esa única variable. Una vez se tienen los datos estructurados, se pasan al segundo modelo. Para este segundo modelo se han usado las mismas arquitecturas que en el Apartado 4.4 y los resultados se encuentran en el Cuadro 12.

<i>EfficientNet</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,5492	0,5492	0,5606	0,5985	0,4962	0,5530
Valor-F	0,6047	0,5704	0,6306	0,6131	0,3636	0,6704
Falsos Positivos	0,2955	0,2500	0,3144	0,2197	0,1477	0,4015
Falsos Negativos	0,1553	0,2008	0,1250	0,1818	0,3561	0,0455
<i>HrNet</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,5909	0,6515	0,6667	0,5303	0,5606	0,5265
Valor-F	0,6327	0,6035	0,6788	0,5921	0,6184	0,6736
Falsos Positivos	0,2614	0,1136	0,1856	0,3106	0,2955	0,4621
Falsos Negativos	0,1477	0,2349	0,1477	0,1591	0,1439	0,0114
<i>ResNet</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,6439	0,6174	0,6326	0,6250	0,5341	0,6136
Valor-F	0,6385	0,6008	0,6394	0,6056	0,4171	0,6394
Falsos Positivos	0,1705	0,1705	0,1932	0,1629	0,1326	0,1932
Falsos Negativos	0,1856	0,2121	0,1742	0,2121	0,3333	0,1742
<i>ConvNext</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,5947	0,5758	0,6439	0,6364	0,4924	0,6288
Valor-F	0,6515	0,5821	0,6569	0,6364	0,5282	0,6525
Falsos Positivos	0,2841	0,2197	0,1970	0,1818	0,2917	0,2197
Falsos Negativos	0,1212	0,2046	0,1591	0,1818	0,2159	0,1515
<i>Deit</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,6250	0,5833	0,6250	0,6477	0,5492	0,6439
Valor-F	0,5751	0,5175	0,6178	0,6437	0,5576	0,6385
Falsos Positivos	0,1288	0,1402	0,1780	0,1705	0,2349	0,1705
Falsos Negativos	0,2462	0,2765	0,1970	0,1818	0,2159	0,1856

Cuadro 12: Resultados del modelo Multi Tarea siguiendo la primera aproximación, con el mejor resultado del Valor-F resaltado en **negrita**.

En el Cuadro 12 se puede ver que los mejores resultados los han dado generalmente los modelos *Random Forest* y *SVM*, concretamente los mejores resultados los han dado con las características extraídas mediante *HrNet*. Si se comparan los resultados con los obtenidos en el Apartado 4.4, al añadir las características de las imágenes se mejoran ligeramente los resultados. Para ver si la dimensión entre las características puede influir en los resultados, se va a probar a normalizar el conjunto de datos y las características antes de hacer la evaluación. Estos resultados se encuentran en el Cuadro 13

<i>EfficientNet</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,6023	0,6364	0,6439	0,6515	0,6553	0,6288
Valor-F	0,4385	0,6444	0,6519	0,6541	0,6459	0,6475
Falsos Positivos	0,0530	0,1932	0,1894	0,1780	0,1591	0,2121
Falsos Negativos	0,3447	0,1705	0,1667	0,1705	0,1856	0,1591
<i>HrNet</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,6174	0,6136	0,6402	0,6061	0,5833	0,5947
Valor-F	0,6300	0,6107	0,6332	0,6031	0,5635	0,6109
Falsos Positivos	0,2083	0,1894	0,1705	0,1932	0,1856	0,2235
Falsos Negativos	0,1742	0,1970	0,1894	0,2008	0,2311	0,1818
<i>ResNet</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,5871	0,5947	0,6288	0,6023	0,5795	0,6023
Valor-F	0,6426	0,6022	0,6231	0,6038	0,5874	0,6209
Falsos Positivos	0,2841	0,2121	0,1780	0,2008	0,2197	0,2235
Falsos Negativos	0,1288	0,1970	0,1932	0,1970	0,2008	0,1742
<i>ConvNext</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,5909	0,6061	0,6402	0,5606	0,5568	0,5909
Valor-F	0,5814	0,6031	0,6388	0,5639	0,5412	0,5878
Falsos Positivos	0,1932	0,1932	0,1780	0,2235	0,2046	0,2008
Falsos Negativos	0,2159	0,2008	0,1818	0,2159	0,2386	0,2083
<i>Deit</i>						
	<i>Árbol C4.5</i>	<i>Árbol CART</i>	<i>Random Forest</i>	<i>Regresión Logística</i>	<i>Red Neuronal</i>	<i>SVM</i>
Exactitud	0,6023	0,5947	0,6402	0,6136	0,6212	0,5682
Valor-F	0,6067	0,5962	0,6304	0,6222	0,6063	0,5615
Falsos Positivos	0,2046	0,2046	0,1667	0,2046	0,1705	0,2083
Falsos Negativos	0,1932	0,2008	0,1932	0,1818	0,2083	0,2235

Cuadro 13: Resultados del modelo Multi Tarea siguiendo la primera aproximación, aplicando normalización, con el mejor resultado del Valor-F resaltado en **negrita**.

Como se puede observar en el Cuadro 13, el mejor resultado lo ha tenido el *Regresión Logística* extrayendo las características con el modelo *EfficientNet*. En general, los mejores resultados los da el modelo *Random Forest*. Si se comparan los resultados con los del Cuadro 12, se observa que los modelos que en este último Cuadro están tanto los resultados más altos como los más bajos. De esto se extrae que normalizar los datos da unos resultados más uniformes.

5.2. Segunda aproximación

La segunda aproximación se basa en resolver varias tareas diferentes con el fin de mejorar las características usadas para la predicción del cáncer. Para este problema, esto se traduce en tomar como entrada del modelo las imágenes de mamografías y que, a partir de la imagen, se devuelvan las predicciones sobre las variables y/o clases de salida, tal y como se puede ver en la figura 14.

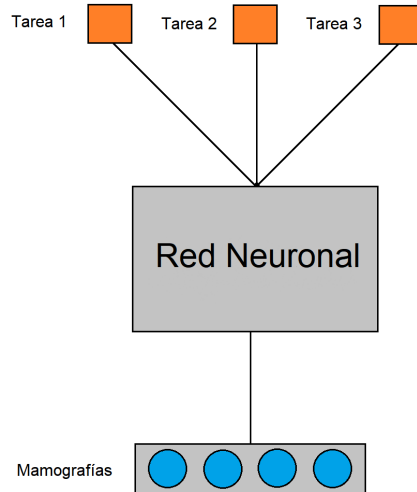


Figura 14: Aproximación 2 para el modelo Multi Tarea. A partir de una entrada, dar varias salidas.

En esta segunda aproximación, además de predecir la variable '*cancer*', también se van a predecir las variables '*age*' y '*density*'. La elección de estas variables es bastante obvia, pues son las dos variables que más influyen en el resultado final, pues una densidad elevada puede ser indicativo de cáncer y dificulta su detección, mientras que la edad también influye, pues las probabilidades de padecer la enfermedad aumentan con los años. También, como se ha visto en los modelos de datos, solamente con esas dos variables se pueden hacer predicciones con buenos resultados, de hecho, las mejores predicciones de entre todas las combinaciones probadas.

Escoger estas variables a priori parece un reto, pues cada una es de un tipo distinto, la variable '*age*' es una variable numérica, la variable '*cancer*' es una variable binaria y la variable '*density*' es una variable categórica. Es por eso por lo que cada una tendrá una métrica distinta y una función de pérdida propia para cada una, que a su vez influirán de manera distinta en la función de pérdida general del modelo.

5.2.1. Descripción de la arquitectura Multi Tarea de la segunda aproximación

Para realizar esta tarea, se ha basado la arquitectura en la desarrollada por T. Dantas en *Medium* [7], donde se desarrolla un modelo de características similares en una versión de *FastAI*.

El primer paso es cómo se van a pasar los datos al modelo, para que éste aprenda de ellos. Para ello se crea una clase que se usará para cumplir dicha función, la llamada *Multi-TaskDataset*, cuyo código se puede ver en la Figura 15. Esta sigue la siguiente estructura:

Inicialización: La clase se crea pasándole un *DataFrame*, que funcionará como base del conjunto de datos, y una lista de transformaciones que se le aplicarán (línea 2). Se recoge la información de cada imagen y cada paciente para guardar las direcciones de las imágenes y poder leerlas más tarde (líneas 4 a 8). También se recogen las variables objetivo (que en un primer momento se han juntado en la variable '*label*') para poder entrenar al modelo y evaluarlo (línea 9). Finalmente, se aplican unas transformaciones de normalización siguiendo las estadísticas de *ImageNet* (línea 13).

Contador de tamaño del conjunto de datos (línea 15).

Selector de objeto (línea 17): Se lee la imagen (línea 19), se le aplican las transformaciones (líneas 20 a 27) y se devuelve la imagen transformada como tensor (línea 34).

Función de muestra, para mostrar los datos y poder comprobar los resultados (líneas 36 a 43).

Como apunte, para la edad se calcula su logaritmo neperiano y se divide por 4,489 (línea 34). El logaritmo neperiano sirve para no dar más importancia a las edades más altas y tomar todos los fallos por igual, y la división por 4,489 se debe a que ese es el máximo valor del logaritmo de la edad (que son 89 años) y de esta manera el valor es un número de entre 0 y 1.

```

1 class MultiTaskDataset():
2     def __init__(self, df, tfms, size=64):
3         self.paths = []
4         for index, row in df.iterrows(): # Se lee el DataFrame
5             patient = row["patient_id"]
6             image = row["image_id"]
7             path = p_under2/str(patient)/(str(image) + '.jpg')
8             self.paths.append(path)
9             self.labels = list(df["label"])
10            self.tfms = tfms
11            self.size = size
12            self.train = df.copy()
13            self.norm = transforms.Normalize([0.485, 0.456, 0.406], [0.229,
0.224, 0.225])
14
15        def __len__(self): return len(self.paths)
16
17        def __getitem__(self, idx):
18            # Se lee la imagen y se transforma
19            img = PIL.Image.open(self.paths[idx]).convert('RGB')
20            img = img.resize((256, 256))
21            t = torch.Tensor(np.array(img))
22            t = t.permute(2, 0, 1).float() / 255.0
23            transform = transforms.ToPILImage()
24            img = PILImage(PILImage.create(transform(t)))
25            img = self.tfms.encode(img)
26            transform = transforms.Compose([transforms.ToTensor()])
27            img = self.norm(transform(img))
28            # Se tratan las variables
29            labels = self.labels[idx].split(" ")
30            age = torch.tensor(float(labels[0]), dtype=torch.float32)
31            cancer = torch.tensor(int(labels[1]), dtype=torch.int64)
32            density = torch.tensor(int(labels[2]), dtype=torch.int64)
33
34            return img, (age.log_() / 4.489, cancer, density)
35
36        def show(self, idx):
37            x, y = self.__getitem__(idx)
38            age, cancer, density = y
39            stds = np.array([0.229, 0.224, 0.225])
40            means = np.array([0.485, 0.456, 0.406])
41            img = ((x.numpy().transpose((1, 2, 0)) * stds + means) * 255).
astype(np.uint8)
42            plt.imshow(img)
43            plt.title("{} {} {}".format(int(age.mul_(4.489).exp_().item()),
cancer.item(), density.item()))

```

Figura 15: Código de la clase *MultiTaskDataset*.

Para el modelo Multi Tarea, también se crea una clase, llamada *MultiTaskModel*, ver Figura 16. Sigue la siguiente estructura:

Inicialización (líneas 2 a 7). Se inicializa siguiendo la misma estructura de la figura 14, con un cuerpo y tres cabezas. El cuerpo puede inicializarse siguiendo cualquier arquitectura de las de *FastAI* (línea 4) y su función es la de extraer características para pasárselas a las tres cabezas, una para cada tarea. A cada tarea se le pasa también el posible número de salidas que puede dar.

Función de paso (líneas 9 a 15). Se lee la información codificada por el modelo y se realizan las predicciones sobre las tres cabezas (líneas 11 a 13). La respuesta final del modelo es una lista con los resultados de cada cabeza (línea 15).

Para la cabeza de la edad, se le ha aplicado una función de sigmoide a su resultado para forzar al modelo a que la salida no se salga del rango de edades (línea 11), es decir, que no dé resultados como por ejemplo puedan ser 116 años, y que como mucho la edad máxima que devuelva sea de 89 años.

```
1 class MultiTaskModel(nn.Module):
2     def __init__(self, arch, ps=0.5):
3         super(MultiTaskModel, self).__init__()
4         self.encoder = create_body(arch)      # Crea un encoder dada una
        arquitectura de FastAI
5         self.fc1 = create_head(1024, 1, ps=ps)
6         self.fc2 = create_head(1024, 2, ps=ps)
7         self.fc3 = create_head(1024, 4, ps=ps)
8
9     def forward(self, x):
10        x = self.encoder(x)
11        age = torch.sigmoid(self.fc1(x))
12        cancer = self.fc2(x)
13        density = self.fc3(x)
14
15        return [age, cancer, density]
```

Figura 16: Código de la clase *MultiTaskModel*.

En cuanto a la métrica, se crea una clase más, llamada *MultiTaskLossWrapper*, ver Figura 17. Sigue la siguiente estructura:

Inicialización (líneas 2 a 5). Se asignan los valores de los parámetros a las estructuras internas de la clase.

Función de paso (líneas 7 a 23). En primer lugar, se calculan los valores de las funciones de pérdida (líneas 10 a 12). Como para cada variable se utiliza una métrica distinta, la función de pérdida ha de ajustarse a cada una. Para la variable ‘age’ se utiliza la raíz del error cuadrático medio, que para tareas de regresión funciona muy bien. Para la variable

‘cancer’ se mantiene el Valor-F, por las mismas razones comentadas con anterioridad. Y para la variable ‘density’ se usa la métrica de Exactitud. Por eso, se utiliza como función de pérdida la función *MSELossFlat* (línea 10) en la variable numérica y *CrossEntropyLossFlat* (líneas 11 y 12) en las binarias y categóricas. El valor óptimo de la función *MSELossFlat* es 0, mientras que para *CrossEntropyLossFlat* ese valor es 1. A cada valor de la función de pérdida se le aplica una ponderación según su métrica (líneas 14 a 21). Finalmente, se devuelve el valor agregado de las funciones de pérdida como valor de la función de pérdida final.

En primera instancia, a todas las funciones de pérdida se les da la misma importancia a la hora de calcular el resultado final (línea 23).

```

1 class MultiTaskLossWrapper(nn.Module):
2     def __init__(self, task_num):
3         super(MultiTaskLossWrapper, self).__init__()
4         self.task_num = task_num
5         self.log_vars = nn.Parameter(torch.zeros((task_num)))
6
7     def forward(self, preds, label):
8         mse, crossEntropy = MSELossFlat(), CrossEntropyLossFlat() #
9         Funciones de perdida
10
11         loss0 = mse(preds[0], label[0])
12         loss1 = crossEntropy(preds[1], label[1])
13         loss2 = crossEntropy(preds[2], label[2])
14
15         precision0 = torch.exp(-self.log_vars[0])
16         loss0 = precision0 * loss0 + self.log_vars[0]
17
18         precision1 = torch.exp(-self.log_vars[1])
19         loss1 = precision1 * loss1 + self.log_vars[1]
20
21         precision2 = torch.exp(-self.log_vars[2])
22         loss2 = precision2 * loss2 + self.log_vars[2]
23
24         return loss0 + loss1 + loss2 # Valor de perdida final

```

Figura 17: Código de la clase *MultiTaskLossWrapper*.

Para evaluar, se aplica de nuevo el callback de *Early Stopping* sobre el Valor-F de la variable ‘cancer’, que es la que más interesa predecir bien.

5.2.2. Resultados de la arquitectura Multi Tarea de la segunda aproximación

Como arquitecturas que se pueden pasar al modelo Multi Tarea no se han tenido en cuenta las de *HrNet w44* y *Deit3 base patch 16 384*, por ser arquitecturas de por sí pesadas y realizar varias tareas sobre un modelo así lo ralentiza aún más. Los resultados de esta arquitectura Multi Tarea con los distintos modelos se pueden observar en el Cuadro 14.

	<i>EfficientNet</i>	<i>Resnet</i>	<i>ConvNext</i>
rmse_age	0,0471	0,3174	0,0650
f1_cancer	0,5733	0,5188	0,6289
acc_density	0,4886	0,3788	0,0644
Tiempo (Minutos)	1	3	4
Época	1	6	3

Cuadro 14: Resultados sobre el conjunto de imágenes balanceado por imágenes sobre un modelo Multi Tarea basado en una de las 3 arquitecturas, con el mejor resultado del Valor-F resaltado en **negrita**.

En el Cuadro 14 se puede ver que los modelos no tienen muchos problemas en predecir la edad, respecto al cáncer se obtiene un Valor-F parecido a los modelos individuales, pero ligeramente inferiores y, en cuanto a la densidad, la Exactitud no es muy alta. El modelo con mejor valor en la métrica de Valor-F sobre la variable ‘cancer’ ha sido *ConvNext*, seguido por *EfficientNet*.

Resulta que, al tener todas las funciones de pérdida con el mismo peso, el modelo intenta ajustar todas por igual, porque se les da a todas la misma importancia. Esto lo que provoca es que el modelo ve como una mejora el conseguir ajustar mejor una variable aun perdiendo ajuste en otras si la mejora de la variable concreta compensa el empeoramiento. Por ejemplo, si en la variable ‘age’ se mejora bastante a costa de empeorar en menor medida los resultados sobre la variable ‘cancer’, el modelo va a intentar seguir ese camino pese a lo que realmente se busca es una mejora en la variable ‘cancer’. Viendo estos resultados, se va a ajustar la función de pérdida para intentar mejorar la predicción de la variable ‘cancer’, asignando pesos de 0,6 al ajuste del cáncer, 0,3 para la densidad y 0,1 para la edad, que es la que mejor calcula (ver Figura 18).

```
1 return 3 * (0.1 * loss0 + 0.6 * loss1 + 0.3 * loss2) # Valor de  
   perdida final
```

Figura 18: Detalle de la declaración de la función de pérdida de la clase *MultiTaskLossWrapper*.

Con estos cambios, los resultados finales están en el Cuadro 15.

	<i>EfficientNet</i>	<i>Resnet</i>	<i>ConvNext</i>
rmse_age	0,4078	0,0419	0,0474
f1_cancer	0,4228	0,6250	0,6305
acc_density	0,1629	0,5492	0,5492
Tiempo (Minutos)	1	3	12
Época	1	7	8

Cuadro 15: Resultados sobre el conjunto de imágenes balanceado por imágenes sobre un modelo Multi Tarea basado en una de las 5 arquitecturas ajustando la función de pérdida, con el mejor resultado del Valor-F resaltado en **negrita**.

Con estos resultados del Cuadro 15, los mejores resultados los da de nuevo el modelo *ConvNext*. Si se comparan los resultados con los vistos en el Cuadro 14, al ajustar la función de pérdida se tiene un Valor-F ligeramente superior. El ajuste de la función de pérdida ha mejorado los resultados en todos los modelos salvo en *EfficientNet*.

6. Conclusiones

En ocasiones gran parte del problema radica en el conjunto de datos. En este caso ha hecho falta realizar trabajos de reducción del tamaño y limpieza del conjunto de datos hasta el punto de que el tamaño del conjunto con el que se ha trabajado es muy inferior al original (un 2,5 % del total). De esto se extrae que el tratamiento sobre el conjunto de datos influye de manera directa a los resultados finales de cualquier estudio, y que de los datos iniciales a los datos finales puede haber grandes cambios.

Los resultados se han evaluado sobre el Valor-F, que, como ya se ha explicado, es la métrica que más se ajusta al rendimiento buscado. Los resultados ordenados son los siguientes: 0,6788 para la primera aproximación Multi Tarea, 0,6742 para el modelo de datos, 0,6703 para el modelo de imágenes y 0,6305 para la segunda aproximación Multi Tarea.

Con estos resultados, se puede observar que no existe una mejora notable al aplicar las técnicas del aprendizaje Multi Tarea en este problema en concreto, pues los resultados son parecidos.

En relación con los resultados, el rendimiento de cada modelo en materia de tiempo también es importante. Realizar las aproximaciones Multi Tarea es un proceso lento. De entre las dos aproximaciones usadas, con una clara diferencia, la aproximación 1 es bastante más pesada, pues requiere del entrenamiento del modelo, la extracción de las características, la fusión de los datos y la evaluación del conjunto fusionado.

Como apertura para una posible continuación de lo realizado en este trabajo, es posible que una aproximación como pueda ser un ensemble sobre los modelos de imagen y datos estructurados pueda ofrecer unos resultados similares, con unos costes de tiempo inferiores a las arquitecturas Multi Tarea.

7. Bibliografía y referencias

Referencias

- [1] *Cáncer de mama: Estadísticas*. Visto el 23-06-2023. 2022. URL: <https://www.cancer.net/es/tipos-de-c%C3%A1ncer/c%C3%A1ncer-de-mama/estad%C3%ADsticas>.
- [2] *FastAI*. 2016. URL: <https://www.fast.ai/>.
- [3] Thiago S Guzella y Walimir M Caminhas. «A review of machine learning approaches to spam filtering». En: *Expert Systems with Applications* 36.7 (2009), págs. 10206-10222.
- [4] Kaiming He et al. «Deep Residual Learning for Image Recognition». En: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Jun. de 2016.
- [5] Brett Koonce y Brett Koonce. «EfficientNet». En: *Convolutional Neural Networks with Swift for Tensorflow: Image Recognition and Dataset Categorization* (2021), págs. 109-123.
- [6] Zhuang Liu et al. «A ConvNet for the 2020s». En: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Jun. de 2022, págs. 11976-11986.
- [7] *Multi-Task Learning with Pytorch and FastAI*. Visto el 28-06-2023. 2020. URL: <https://towardsdatascience.com/multi-task-learning-with-pytorch-and-fastai-6d10dc7ce855>.
- [8] Radiological Society of North American. *RSNA Screening Mammography Breast Cancer Detection*. 2022. URL: <https://kaggle.com/competitions/rsna-breast-cancer-detection>.
- [9] *NumPy*. 1995. URL: <https://numpy.org/>.
- [10] Sociedad Española de Oncología Médica (SEOM). *Las cifras del cáncer en España 2020*. Página 6.
- [11] *Pandas*. 2022. URL: <https://pandas.pydata.org/>.
- [12] *PyTorch*. 2017. URL: <https://pytorch.org/>.
- [13] *scikit-learn*. 2010. URL: <https://scikit-learn.org/>.
- [14] Ray Smith. «An overview of the Tesseract OCR engine». En: *Ninth international conference on document analysis and recognition (ICDAR 2007)*. Vol. 2. IEEE. 2007, págs. 629-633.
- [15] *timm*. 2022. URL: <https://timm.fast.ai/>.
- [16] *Torch*. 2002. URL: <http://torch.ch/>.
- [17] Hugo Touvron et al. «Training data-efficient image transformers & distillation through attention». En: *International conference on machine learning*. PMLR. 2021, págs. 10347-10357.

- [18] Jingdong Wang et al. «Deep high-resolution representation learning for visual recognition». En: *IEEE transactions on pattern analysis and machine intelligence* 43.10 (2020), págs. 3349-3364.
- [19] Wikipedia. *Aprendizaje automático* — *Wikipedia, La enciclopedia libre*. [Visto el 26-06-2023]. 2023. URL: https://es.wikipedia.org/w/index.php?title=Aprendizaje_autom%C3%A1tico&oldid=151715205.
- [20] Wikipedia. *Aprendizaje profundo* — *Wikipedia, La enciclopedia libre*. [Visto el 26-06-2023]. 2023. URL: https://es.wikipedia.org/w/index.php?title=Aprendizaje_profundo&oldid=152070310.
- [21] Wikipedia. *Cáncer de mama* — *Wikipedia, La enciclopedia libre*. [Visto el 23-06-2023]. 2023. URL: https://es.wikipedia.org/w/index.php?title=C%C3%A1ncer_de_mama&oldid=151946076.
- [22] Wikipedia. *Inteligencia artificial* — *Wikipedia, La enciclopedia libre*. [Visto el 26-06-2023]. 2023. URL: https://es.wikipedia.org/w/index.php?title=Inteligencia_artificial&oldid=152179191.
- [23] Anbang Xu et al. «A new chatbot for customer service on social media». En: *Proceedings of the 2017 CHI conference on human factors in computing systems*. 2017, págs. 3506-3510.

A. Descripción del conjunto de datos

En esta Sección se describen los conjuntos de datos, para que se puedan consultar y usar como referencia. También así se pueden entender mejor en su conjunto, pues se ve cómo están distribuidos.

A.1. Descripción del conjunto completo

El conjunto de datos representa la información de las 54.706 imágenes del conjunto completo, mediante una serie de variables o atributos, siendo éstas y su distribución la siguiente:

- **‘site_id’**: El identificador del hospital de origen, en el que se realizaron las imágenes y se recogieron los datos. Hay 2 hospitales distintos desde los que provienen las imágenes.
 - Hospital 1: 29.519 imágenes (54 %).
 - Hospital 2: 25.187 imágenes (46 %).
- **‘patient_id’**: El identificador del paciente. En total hay 11.913 pacientes distintos, cada uno con sus imágenes.
- **‘image_id’**: El identificador de la imagen. Cada fila del conjunto de datos representa a una imagen. Hay 54.706 imágenes en total, y no hay ninguna repetida.
- **‘laterality’**: Indica si la imagen se ha tomado sobre el pecho izquierdo (L) o sobre el derecho (R). Hay casi las mismas imágenes para cada lado.
 - Lado derecho (R): 27.439 imágenes (50 %).
 - Lado izquierdo (L): 27.267 imágenes (50 %).
- **‘view’**: Indica la orientación de la imagen. El procedimiento por defecto es capturar dos vistas por cada pecho. Hay dos vistas muy usadas, con números muy parecidos, y otras vistas muy poco usadas. Esto es normal, pues las vistas *MLO* y *CC* son las vistas estándar.
 - *MLO* (Oblicua Mediolateral): 27.903 imágenes (52 %).
 - *CC* (Craneocaudal): 26.765 imágenes (48 %).
 - *AT* (Cola Axilar): 19 imágenes (0 %).
 - *LM* (Lateromedial): 10 imágenes (0 %).
 - *ML* (Mediolateral): 8 imágenes (0 %).
 - *LMO* (Oblicua Lateromedial): 1 imagen (0 %).

- **‘age’**: Indica la edad del paciente en años. Distribución con forma normal centrada en los 58 años. Las edades varían desde los 26 años hasta los 89. Hay 37 instancias con valores faltantes (*NaN*).
- **‘implant’**: Indica si el paciente tiene implantes de pecho. El hospital con el identificador 1 da la información de los implantes a nivel de paciente, no de pecho, por lo que en este caso no se puede determinar con certeza que el implante sea un implante de pecho. La clase sin implantes tiene 53.229 imágenes (97 %) y la clase con implantes tiene 1.477 imágenes (el 3 % restante).
- **‘density’**: Es una puntuación que indica lo denso que es el tejido del pecho, asignando A para la menor densidad y D para la mayor. Se indica que un tejido más denso puede dificultar el diagnóstico. Esta variable sólo aparece en el conjunto de Entrenamiento. Hay 4 clases que siguen una distribución normal:
 - *NaN*: Hay bastantes entradas sin valor, concretamente 25.236, lo que representa un 46 % del total.
 - *A*: 3.105 entradas (6 % del total).
 - *B*: 12.651 entradas (23 %).
 - *C*: 12.175 entradas (22 %).
 - *D*: 1.539 entradas (3 %).
- **‘machine_id’**: Un identificador para la máquina que sacó la imagen. Se han usado 10 máquinas para realizar las mamografías.
 - Máquina 21: 8.221 imágenes (15 %).
 - Máquina 29: 8.267 imágenes (15 %).
 - Máquina 48: 8.699 imágenes (16 %).
 - Máquina 49: 23.529 imágenes (43 %).
 - Máquina 93: 1.915 imágenes (3,35 %).
 - Máquina 170: 923 imágenes (2 %).
 - Máquina 190: 145 imágenes (0,3 %).
 - Máquina 197: 29 imágenes (0,0005 %).
 - Máquina 210: 1.070 imágenes (2 %).
 - Máquina 216: 1.908 imágenes (3,35 %).
- **‘cancer’**: Es la variable objetivo, indica si el pecho ha dado positivo en cáncer o no. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (la que no tiene cáncer), tiene 53.548 imágenes (98 %) y la clase positiva tiene 1.158 imágenes (2 %).

- **‘biopsy’**: Indica si una biopsia se ha realizado en el pecho. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (la que no tiene biopsia) tiene 51.737 imágenes (94,5 %) y la clase positiva tiene 2.969 imágenes (5,5 %).
- **‘invasive’**: En caso de detectar cáncer, indica si el mismo es invasivo o no. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (sin cáncer invasivo) tiene 53.888 imágenes (98,5 %) y la clase positiva tiene 818 imágenes (1,5 %).
- **‘BIRADS’**: 3 clases.
 - *NaN*: 28.420 imágenes sin valor asociado (52 %).
 - *0*: El pecho necesita seguimiento en 8.249 imágenes (15 %).
 - *1*: El pecho ha dado negativo en cáncer en 15.772 imágenes (29 %).
 - *2*: El pecho es normal en 2.265 imágenes (4 %).
- **‘prediction_id’**: El identificador para la fila que se corresponde con la predicción. Esta variable sólo aparece en el conjunto de Prueba.
- **‘difficult_negative_case’**: Tiene valor positivo si el caso fue más difícil de lo habitual. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa tiene 47.001 entradas (86 %) y la positiva 7.705 (14 %).

Es curioso que haya más imágenes con biopsia que con cáncer, entonces es de suponer que la biopsia se ha realizado antes que la imagen, y puede ser que sean pacientes que hayan tenido cáncer de mama en el pasado, pero que actualmente no lo tienen.

A.2. Descripción del conjunto tras la limpieza

El conjunto de datos representa la información de las 29.443 imágenes del conjunto tras la limpieza, mediante una serie de variables o atributos, siendo éstas y su distribución la siguiente:

- **‘site_id’**: El identificador del hospital de origen, en el que se realizaron las imágenes y se recogieron los datos. Hay 2 hospitales distintos desde los que provienen las imágenes.
 - Hospital 1: 29.433 imágenes (100 %).
- **‘patient_id’**: El identificador del paciente. En total hay 5.801 pacientes distintos, cada uno con sus imágenes.
- **‘image_id’**: El identificador de la imagen. Cada fila del conjunto de datos representa a una imagen. Hay 29.433 imágenes en total, y no hay ninguna repetida.
- **‘laterality’**: Indica si la imagen se ha tomado sobre el pecho izquierdo (L) o sobre el derecho (R). Hay casi las mismas imágenes para cada lado.

- Lado derecho (R): 14.803 imágenes (50 %).
- Lado izquierdo (L): 14.630 imágenes (50 %).
- **‘view’**: Indica la orientación de la imagen. El procedimiento por defecto es capturar dos vistas por cada pecho. Hay dos vistas muy usadas, con números muy parecidos, y otras vistas muy poco usadas. Esto es normal, pues las vistas MLO y CC son las vistas estándar.
 - MLO (Oblicua Mediolateral): 15.121 imágenes (52 %).
 - CC (Craneocaudal): 14.279 imágenes (48 %).
 - AT (Cola Axilar): 19 imágenes (0 %).
 - LM (Lateromedial): 5 imágenes (0 %).
 - ML (Mediolateral): 8 imágenes (0 %).
 - LMO (Oblicua Lateromedial): 1 imagen (0 %).
- **‘age’**: Indica la edad del paciente en años. Distribución con forma normal centrada en los 58 años. Las edades varían desde los 26 años hasta los 89.
- **‘implant’**: Indica si el paciente tiene implantes de pecho. El hospital con el identificador 1 (que es el único en este conjunto) da la información de los implantes a nivel de paciente, no de pecho, por lo que en este caso no se puede determinar con certeza que el implante sea un implante de pecho. La clase sin implantes tiene 27.973 imágenes (95 %) y la clase con implantes tiene 1.460 imágenes (el 5 % restante).
- **‘density’**: Es una puntuación que indica lo denso que es el tejido del pecho, asignando A para la menor densidad y D para la mayor. Se indica que un tejido más denso puede dificultar el diagnóstico. Esta variable sólo aparece en el conjunto de Entrenamiento. Hay 4 clases que siguen una distribución normal:
 - A: 3.105 (11 %).
 - B: 12.642 (43 %).
 - C: 12.155 (41 %).
 - D: 1.531 (5 %).

- ***'machine_id'***: Un identificador para la máquina que sacó la imagen. Se han usado 10 máquinas para realizar las mamografías.
 - Máquina 49: 23.451 imágenes (80 %).
 - Máquina 93: 1.915 imágenes (6,5 %).
 - Máquina 170: 923 imágenes (3 %).
 - Máquina 190: 145 imágenes (0,5 %).
 - Máquina 197: 29 imágenes (0,0001 %).
 - Máquina 210: 1.070 imágenes (3,5 %).
 - Máquina 216: 1.908 imágenes (6,5 %).
- ***'cancer'***: Es la variable objetivo, indica si el pecho ha dado positivo en cáncer o no. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (la que no tiene cáncer), tiene 28.769 imágenes (98 %) y la clase positiva tiene 664 imágenes (2 %).
- ***'biopsy'***: Indica si una biopsia se ha realizado en el pecho. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (la que no tiene biopsia) tiene 27.080 imágenes (92 %) y la clase positiva tiene 2.353 imágenes (8 %).
- ***'invasive'***: En caso de detectar cáncer, indica si el mismo es invasivo o no. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (sin cáncer invasivo) tiene 28.983 imágenes (98,5 %) y la clase positiva tiene 450 imágenes (1,5 %).
- ***'BIRADS'***: 3 clases.
 - *NaN*: 5.796 imágenes sin valor asociado (20 %).
 - 0: El pecho necesita seguimiento en 5.664 imágenes (19 %).
 - 1: El pecho ha dado negativo en cáncer en 15.734 imágenes (53 %).
 - 2: El pecho es normal en 2.249 imágenes (8 %).
- ***'prediction_id'***: El identificador para la fila que se corresponde con la predicción. Esta variable sólo aparece en el conjunto de Prueba.
- ***'difficult_negative_case'***: Tiene valor positivo si el caso fue más difícil de lo habitual. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa tiene 24.433 entradas (83 %) y la positiva 5.000 (7 %).

En el caso de la variable *'biopsy'* sigue ocurriendo lo mismo que en el apartado anterior. También, se puede ver que solamente queda el hospital 1, por lo que la variable *'site_id'* y la variable *'implant'* se pueden eliminar, ya que ésta última no es muy fiable si el dato proviene del hospital en cuestión.

A.3. Descripción del conjunto balanceado

El conjunto de datos representa la información de las 1.320 imágenes del conjunto balanceado, mediante una serie de variables o atributos, siendo éstas y su distribución la siguiente:

- **'patient_id'**: El identificador del paciente. En total hay 863 pacientes distintos, cada uno con sus imágenes.
- **'image_id'**: El identificador de la imagen. Cada fila del conjunto de datos representa a una imagen. Hay 1.320 imágenes en total, y no hay ninguna repetida.
- **'laterality'**: Indica si la imagen se ha tomado sobre el pecho izquierdo (L) o sobre el derecho (R). Hay casi las mismas imágenes para cada lado.
 - Lado derecho (*R*): 644 imágenes (49 %).
 - Lado izquierdo (*L*): 676 imágenes (51 %).
- **'view'**: Indica la orientación de la imagen. El procedimiento por defecto es capturar dos vistas por cada pecho. Hay dos vistas muy usadas, con números muy parecidos, y otras vistas muy poco usadas. Esto es normal, pues las vistas *MLO* y *CC* son las vistas estándar.
 - *MLO* (Oblicua Mediolateral): 691 imágenes (52,5 %).
 - *CC* (Craneocaudal): 626 imágenes (47,5 %).
 - *AT* (Cola Axilar): 2 imágenes (0,002 %).
 - *LM* (Lateromedial): 1 imagen (0,001 %).
- **'age'**: Indica la edad del paciente en años. Distribución con forma normal centrada en los 61 años. Las edades varían desde los 35 años hasta los 89.
- **'density'**: Es una puntuación que indica lo denso que es el tejido del pecho, asignando A para la menor densidad y D para la mayor. Se indica que un tejido más denso puede dificultar el diagnóstico. Esta variable sólo aparece en el conjunto de Entrenamiento. Hay 4 clases que siguen una distribución normal:
 - *A*: 126 (10 %).
 - *B*: 560 (42,5 %).
 - *C*: 567 (42,5 %).
 - *D*: 67 (5 %).

- ***'machine_id'***: Un identificador para la máquina que sacó la imagen. Se han usado 10 máquinas para realizar las mamografías.
 - Máquina 49: 1.129 imágenes (85,5 %).
 - Máquina 93: 64 imágenes (5 %).
 - Máquina 170: 46 imágenes (3,5 %).
 - Máquina 190: 5 imágenes (0,004 %).
 - Máquina 210: 15 imágenes (1 %).
 - Máquina 216: 61 imágenes (5 %).
- ***'cancer'***: Es la variable objetivo, indica si el pecho ha dado positivo en cáncer o no. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (la que no tiene cáncer), tiene 656 imágenes (50 %) y la clase positiva tiene 664 imágenes (50 %).
- ***'biopsy'***: Indica si una biopsia se ha realizado en el pecho. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (la que no tiene biopsia) tiene 623 imágenes (47 %) y la clase positiva tiene 697 imágenes (53 %).
- ***'invasive'***: En caso de detectar cáncer, indica si el mismo es invasivo o no. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa (sin cáncer invasivo) tiene 870 imágenes (66 %) y la clase positiva tiene 450 imágenes (34 %).
- ***'BIRADS'***: 3 clases.
 - *NaN*: 118 imágenes sin valor asociado (8 %).
 - 0: El pecho necesita seguimiento en 775 imágenes (59 %).
 - 1: El pecho ha dado negativo en cáncer en 386 imágenes (30 %).
 - 2: El pecho es normal en 41 imágenes (3 %).
- ***'prediction_id'***: El identificador para la fila que se corresponde con la predicción. Esta variable sólo aparece en el conjunto de Prueba.
- ***'difficult_negative_case'***: Tiene valor positivo si el caso fue más difícil de lo habitual. Esta variable sólo aparece en el conjunto de Entrenamiento. La clase negativa tiene 1.209 entradas (81,5 %) y la positiva 111 (8,5 %).

En el caso de la variable *'biopsy'* sigue ocurriendo lo mismo que en los apartados anteriores.

B. Resultados sobre el balanceo por pacientes

En este apartado se muestran los resultados obtenidos con el conjunto de datos balanceado por pacientes, con el objetivo de poder compararlos con los resultados finales. Los parámetros de evaluación son los mismos que en el Apartado 4.1, recalculando para cada modelo el ratio de aprendizaje óptimo y aumentando el número máximo de épocas a 20. Estos resultados están en el Cuadro 16.

	<i>EfficientNet</i>	<i>HrNet</i>	<i>ResNet</i>	<i>ConvNext</i>	<i>Deit</i>
Exactitud	0,5973	0,5395	0,6301	0,5549	0,6879
Precisión	0,2951	0,2667	0,3274	0,2988	0,3542
Recuperación	0,4030	0,4478	0,4105	0,5373	0,2537
Valor-F	0,3407	0,3343	0,3642	0,3840	0,2957
Tiempo (Minutos)	1	10	4	4	3
Época	1	5	6	4	2

Cuadro 16: Resultados de las 5 arquitecturas sobre el conjunto de imágenes balanceado por pacientes, con el mejor resultado del Valor-F resaltado en **negrita**.

Al evaluar sobre el balanceo por pacientes, se tiene un volumen de datos muy inferior al anterior, por lo que los tiempos son bastante inferiores y, por tanto, se pueden entrenar durante más épocas. En el Cuadro 16, se puede ver una mejora respecto al conjunto completo del Cuadro 3. Sin embargo, son inferiores a los del Cuadro 4, debido a que balancear por pacientes no equilibra las clases. Esto lo que demuestra es que cuánto más cercanas están las clases de tener el mismo número de instancias, mejores resultados dan los modelos y, por tanto, el mejor conjunto es el balanceado por imágenes. Para comparar también este balanceo con los resultados de los otros conjuntos de datos, se puede comparar la Matriz de Confusión (Figura 19).

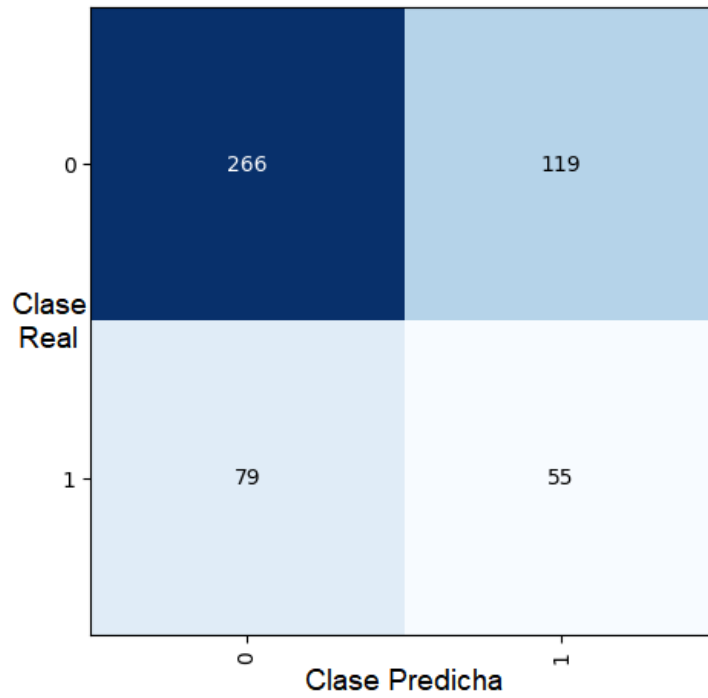


Figura 19: Visualización de la matriz de confusión sobre el conjunto balanceado por pacientes (concretamente la del modelo *ConvNext*).

En la Figura 19, se puede ver cómo se empieza a realizar predicciones sobre la clase positiva. A diferencia de lo que ocurría con el conjunto completo.

C. Ajuste de parámetros para los modelos de datos

Para el ajuste por parámetros, en las pruebas se ha utilizado un *param grid*, de forma que se prueban combinaciones entre los valores de los parámetros para buscar el modelo que dé el mejor resultado. Para todos los modelos, ya que tienen una parte estocástica, se ha asignado un valor de semilla aleatoria de 0. Para los modelos, estos parámetros son los siguientes:

- **Árbol C4.5.**
 - *criterion* = ‘entropy’. (Valor por defecto del Árbol C4.5)
 - *max_depth* = 5, 10, None.
 - *min_samples_split* = 2, 10, 20.

- **Árbol CART.**
 - *criterion* = ‘gini’. (Valor por defecto del *Árbol CART*)
 - *max_depth* = 5, 10, None.
 - *min_samples_split* = 2, 10, 20.
- **Random Forest.**
 - *criterion* = ‘gini’, ‘entropy’.
 - *max_features* = ‘sqrt’, ‘log2’, None.
 - *bootstrap* = True, False.
- **Regresión Logística.**
 - *C* = 0,001, 0,01, 0,1, 1.
- **Red Neuronal.**
 - *hidden_layer_sizes* = 10, 25, 50, 100.
 - *alpha* = 0,0001, 0,001, 0,01, 0,1.
- **SVM.**
 - *gamma* = ‘auto’ (Valor por defecto).
 - *decision_function_shape* = ‘ovo’, ‘ovr’.

Los parámetros en cada una de las pruebas se consiguen mediante la evaluación sobre el conjunto de validación. Para el conjunto de datos final, estos son los valores de los parámetros:

- **Árbol C4.5.**
 - *criterion* = ‘entropy’. (Valor por defecto del *Árbol C4.5*)
 - *max_depth* = None.
 - *min_samples_split* = 2.
- **Árbol CART.**
 - *criterion* = ‘gini’. (Valor por defecto del *Árbol CART*)
 - *max_depth* = None.
 - *min_samples_split* = 2.
- **Random Forest.**
 - *criterion* = ‘entropy’.
 - *max_features* = None.
 - *bootstrap* = True.

- *Regresión Logística.*
 - $C = 0,01$.
- *Red Neuronal.*
 - $hidden_layer_sizes = 50$.
 - $alpha = 0,1$.
- *SVM.*
 - $gamma = 'auto'$ (Valor por defecto).
 - $decision_function_shape = 'ovo'$.

D. Modelo Multi Tarea, Aproximación 2 con función de pérdida única

Viendo los resultados del Apartado 5.2.2, se va a probar a cambiar la función de pérdida del modelo Multi Tarea teniendo en cuenta únicamente la pérdida asociada a la variable ‘*cancer*’, para ver cómo afecta al resultado del modelo. Esto teóricamente no tendría mucho sentido, pues al hacer esto, el modelo sólo se fijaría en las características relacionadas con esa variable, obviando aquellas de las otras dos variables y haciendo que el Multi Tarea pierda su sentido. Sin embargo, en la práctica puede que esto sea diferente. Los resultados están en el Cuadro 17.

	<i>EfficientNet</i>	<i>Resnet</i>	<i>ConvNext</i>
rmse_age	0,5169	0,4313	0,4420
f1_cancer	0,5829	0,6326	0,5698
acc_density	0,1212	0,1970	0,1856
Tiempo (Minutos)	3	2,5	11
Época	4	5	7

Cuadro 17: Resultados sobre el conjunto de imágenes balanceado por imágenes sobre sobre un modelo Multi Tarea basado en una de las 3 arquitecturas teniendo en cuenta únicamente la variable ‘*cancer*’ para la función de pérdida, con el mejor resultado del Valor-F resaltado en **negrita**.

Se puede ver en el Cuadro 17 que los resultados se corresponden con la teoría, y los resultados son peores que los del modelo Multi Tarea con la función de pérdida ajustada. Es notable especialmente en las variables ‘*age*’ y ‘*density*’ el empeoramiento de los resultados. Como teóricamente se ha dicho, al no tener tanto contexto y no fijarse en la información de las otras dos variables, da unos resultados similares a los del modelo individual. Es por esto por lo que una función de pérdida teniendo en cuenta solamente la variable ‘*cancer*’ no tiene sentido.